

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ
«ТЕХНОЛОГІЧНИЙ ФАХОВИЙ КОЛЕДЖ
НАЦІОНАЛЬНОГО УНІВЕРСИТЕТУ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»**

**ПОЯСНЮВАЛЬНА ЗАПИСКА
ДО КВАЛІФІКАЦІЙНОЇ РОБОТИ**

фахового молодшого бакалавра
галузі знань 12 «Інформаційні технології»
спеціальності 122 «Комп'ютерні науки»

освітньо-професійної програми «Комп'ютерні науки»

на тему: **«Розроблення голосового асистента для ПК з інтегрованим
модулем штучного інтелекту для WEB-пошуку з підтримкою української
мови»**

Здобувач освіти:

Заваринський Андрій Андрійович

Навчальна група КН-41

Керівник кваліфікаційної роботи:

Климкович Тамара Анатоліївна

Рецензент кваліфікаційної роботи:

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ
«ТЕХНОЛОГІЧНИЙ ФАХОВИЙ КОЛЕДЖ
НАЦІОНАЛЬНОГО УНІВЕРСИТЕТУ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»

СХВАЛЕНО

Голова циклової комісії
Інформаційно-комп'ютерної підготовки
_____ О. Ковтонюк

Протокол № _____ 7
від « 10 » березня 2026 р.

ЗАТВЕРДЖУЮ

Заступник директора
з навчальної роботи
_____ М. Підвальний
18 березня 2026 р.

ЗАВДАННЯ
НА КВАЛІФІКАЦІЙНУ РОБОТУ

фахового молодшого бакалавра
галузі знань 12 «Інформаційні технології»
спеціальності 122 «Комп'ютерні науки»
освітньо-професійної програми «Комп'ютерні науки»

Здобувач освіти: **Заваринський Андрій Андрійович**
Навчальна група: **КН-41**
Тема: **Розроблення голосового асистента для ПК з
інтегрованим модулем штучного інтелекту для
WEB-пошуку з підтримкою української мови**
Керівник: **Климович Тамара Анатоліївна**

Вихідні дані:

Розробити програму у вигляді голосового асистента для ПК, що забезпечує пошук інформації в Інтернеті за голосовими командами.

Передбачити підтримку української мови з розпізнаванням мовлення та формуванням текстових і голосових відповідей.

Програма має обробляти голосові команди, виконувати пошук, озвучувати результати та здійснювати базові дії операційної системи.

Необхідно створити простий графічний інтерфейс для керування, відображення результатів і статусу.

Забезпечити роботу в Windows із використанням штучного інтелекту для обробки команд.

Зміст пояснювальної записки:

Титульний аркуш
Завдання на кваліфікаційну роботу
Анотація українською мовою
Анотація англійською мовою
Зміст

Вступ

Розділ 1. Аналіз завдання

- 1.1. Обґрунтування актуальності розробки голосового асистента
- 1.2. Вибір засобів розробки

Розділ 2. Проектування голосового асистента

- 2.1. Проектування архітектури програми
- 2.2. Проектування інтерфейсу користувача
- 2.3. Вибір алгоритмів для розпізнавання та обробки голосу

Розділ 3. Розроблення голосового асистента

- 3.1. Реалізація модуля розпізнавання голосу
- 3.2. Реалізація модуля обробки запитів та пошуку інформації
- 3.3. Інтеграція голосових відповідей та виводу результатів

Розділ 4. Експериментальна частина

- 4.1. Тестування роботи голосового асистента
- 4.2. Аналіз отриманих результатів

Розділ 5. Економічна частина

- 5.1. Вибір та обґрунтування базового програмного продукту
- 5.2. Розрахунок собівартості нового програмного продукту та витрат на виконання кваліфікаційної роботи
- 5.3. Визначення ціни програмного продукту
- 5.4. Висновки

Розділ 6. Охорона праці

- 6.1. Характеристика робочого місця програміста
- 6.2. Техніка безпеки при роботі з ПК
- 6.3. Промислова санітарія
- 6.4. Вимоги пожежної безпеки

Висновки

Список використаних джерел

Додатки

Перелік графічного матеріалу:

- 1. Програма “Голосовий асистент”. Структура роботи голосового асистента
- 2. Програма “Голосовий асистент”. Структура взаємодії основних модулів програми

Консультанти розділів роботи:

Економічна частина	– Комар І.О.
Охорона праці	– Боднар М.А.

Термін перевірки текстової частини на унікальність – 07.06.2026 р.

Термін здачі кваліфікаційної роботи до захисту – 09.06.2026 р.

Здобувач освіти

_____ **А. Заваринський**
(підпис)

Керівник роботи

_____ **Т. Климкович**
(підпис)

АНОТАЦІЯ

Заваринський А. А. Розроблення голосового асистента для ПК з інтегрованим модулем штучного інтелекту для WEB-пошуку з підтримкою української мови: кваліфікаційна робота / Заваринський А. А.; керівник Климкович Т. А. — Львів: ВСП «Технологічний фаховий коледж НУ «Львівська політехніка», 2026. — 78 с.

Кваліфікаційна робота присвячена розробці україномовного голосового асистента «Sophia» для операційної системи Windows. У роботі проведено аналіз існуючих голосових асистентів та засобів їх розробки, обґрунтовано вибір інструментарію та спроектовано архітектуру програмного продукту.

Програмний продукт розроблено мовою програмування Python 3.12 з графічним інтерфейсом на базі бібліотеки CustomTkinter. Реалізовано модуль розпізнавання голосу засобами бібліотек SpeechRecognition та Vosk (офлайн-режим), модуль синтезу мовлення на базі Microsoft Edge-TTS (голос uk-UA-PolinaNeural), модуль обробки природної мови на основі Mistral AI API, а також модулі отримання актуальних даних (погода, новини).

Проведено тестування роботи асистента та аналіз отриманих результатів. Виконано техніко-економічне обґрунтування розробки: собівартість програмного продукту становить 9 143,50 грн, відпускна ціна з ПДВ — 13 715 грн. Розроблено заходи з охорони праці на робочому місці програміста.

Ключові слова: голосовий асистент, розпізнавання мовлення, синтез мовлення, Python, штучний інтелект, Vosk, Edge-TTS, Mistral AI, офлайн-режим, CustomTkinter.

ANNOTATION

Zavarynsky A. A. Development of a voice assistant for PCs with an integrated artificial intelligence module for WEB search with support for the Ukrainian language: qualification thesis / A. A. Zavarynsky; supervisor Klymkovych T. A. — Lviv: VSP “Technological Professional College of NU “Lviv Polytechnic”, 2026. — 78 p.

The qualification thesis is dedicated to the development of the Ukrainian-language voice assistant “Sophia” for the Windows operating system. The work includes an analysis of existing voice assistants and their development tools, justification of the chosen technology stack, and the design of the software product architecture.

The software product was developed in Python 3.12 with a graphical user interface based on the CustomTkinter library. The following modules were implemented: voice recognition using SpeechRecognition and Vosk (offline mode), speech synthesis via Microsoft Edge-TTS (uk-UA-PolinaNeural voice), natural language processing powered by Mistral AI API, and retrieval of real-time data (weather, news).

Testing of the assistant and analysis of the obtained results were conducted. A techno-economic justification was performed: the cost price of the software product is UAH 9,143.50 and the retail price including VAT is UAH 13,715. Labor protection measures for the programmer’s workplace were developed.

Keywords: voice assistant, speech recognition, speech synthesis, Python, artificial intelligence, Vosk, Edge-TTS, Mistral AI, offline mode, CustomTkinter.

ЗМІСТ

ВСТУП.....	7
РОЗДІЛ 1. АНАЛІЗ ЗАВДАННЯ.....	12
1.1. Обґрунтування актуальності розробки голосового асистента.....	12
1.2. Вибір засобів розробки.....	16
РОЗДІЛ 2. ПРОЄКТУВАННЯ ГОЛОСОВОГО АСИСТЕНТА.....	18
2.1. Проєктування архітектури програми.....	18
2.2. Проєктування інтерфейсу користувача.....	21
2.3. Вибір алгоритмів для розпізнавання та обробки голосу.....	24
РОЗДІЛ 3. РОЗРОБЛЕННЯ ГОЛОСОВОГО АСИСТЕНТА.....	30
3.1. Реалізація модуля розпізнавання голосу.....	30
3.2. Реалізація модуля обробки запитів та пошуку інформації.....	34
3.3. Інтеграція голосових відповідей та виводу результатів.....	39
РОЗДІЛ 4. ЕКСПЕРИМЕНТАЛЬНА ЧАСТИНА.....	44
4.1. Тестування роботи голосового асистента.....	44
4.2. Аналіз отриманих результатів.....	49
РОЗДІЛ 5. ЕКОНОМІЧНА ЧАСТИНА.....	52
5.1. Вибір та обґрунтування базового програмного продукту.....	52
5.2. Розрахунок собівартості нового програмного продукту та витрат на виконання кваліфікаційної роботи.....	55
5.3. Визначення ціни програмного продукту.....	59
5.4. Висновки.....	60
РОЗДІЛ 6. ОХОРОНА ПРАЦІ.....	62
6.1. Характеристика робочого місця програміста.....	62
6.2. Техніка безпеки при роботі з ПК.....	62
6.3. Промислова санітарія.....	64
6.4. Вимоги пожежної безпеки.....	66
ВИСНОВКИ.....	71
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	74
ДОДАТКИ.....	76

					КР. 122. КН-41. 05. 26. ПЗ.				
Зм	Арк	№ докум.	Підпис	Дата					
Розробив		А. Заваринський			Розроблення голосового асистента для ПК з інтегрованим модулем штучного інтелекту для WEB-пошуку з підтримкою української мови	Літера		Аркуш	Аркушів
Перевірив		Т. Климкович					Н	7	
Консультант		І. Комар				ВСП «ТФКНУ«ЛП»			
Консультант.		Боднар М.А.							
Рецензент									

ВСТУП

У сучасних умовах розвитку інформаційних технологій дедалі більшого значення набувають засоби природної взаємодії людини з комп'ютером. Якщо раніше основними способами керування персональним комп'ютером були клавіатура, миша та графічний інтерфейс, то сьогодні активно розвиваються голосові інтерфейси, які дають змогу виконувати команди, здійснювати пошук інформації та отримувати відповіді за допомогою усного мовлення. Такі технології поступово стають звичними у смартфонах, побутових пристроях, автомобільних системах, сервісах розумного дому та персональних комп'ютерах.

Голосові асистенти є прикладом програмних систем, які поєднують розпізнавання мовлення, обробку природної мови, синтез голосу та виконання команд користувача. Вони спрощують взаємодію з комп'ютерними системами, дозволяють автоматизувати рутинні дії, швидко отримувати інформацію з Інтернету, відкривати програми, керувати мультимедіа та виконувати інші повсякденні завдання. Особливо важливими такі системи є для користувачів, яким складно або незручно постійно використовувати клавіатуру чи мишу.

Актуальність розроблення голосового асистента з підтримкою української мови зумовлена кількома чинниками. По-перше, зростає потреба у створенні якісних україномовних програмних продуктів, які були б зручними для широкого кола користувачів. По-друге, стрімко розвиваються технології штучного інтелекту, розпізнавання мовлення та синтезу голосу, що відкриває можливості для створення більш гнучких і функціональних асистентів. По-третє, персональні комп'ютери залишаються важливим інструментом навчання, роботи та повсякденної діяльності, тому голосове керування ними може суттєво підвищити зручність використання.

Незважаючи на наявність відомих комерційних голосових асистентів, таких як Google Assistant, Apple Siri, Amazon Alexa та Microsoft Cortana, їх використання для україномовних користувачів персональних комп'ютерів має

низку обмежень. Частина таких систем орієнтована переважно на мобільні пристрої або пристрої розумного дому, частина не забезпечує повноцінної підтримки української мови, а деякі рішення втратили актуальність або були зняті з підтримки. Наприклад, підтримку Microsoft Cortana було припинено, Apple Siri не має повноцінної україномовної підтримки, а Amazon Alexa здебільшого використовується в межах власної екосистеми пристроїв. У результаті виникає потреба у створенні окремого рішення, орієнтованого саме на роботу в операційній системі Windows та взаємодію українською мовою.

Додатковим чинником актуальності є загальна цифровізація суспільства. В Україні активно розвиваються електронні сервіси, онлайн-освіта, дистанційна робота, цифрові інструменти комунікації та державні електронні послуги. У таких умовах зростає значення програмних продуктів, які роблять взаємодію з комп'ютером простішою, доступнішою та зрозумілішою. Голосовий інтерфейс може стати зручним інструментом як для досвідчених користувачів, так і для людей з нижчим рівнем комп'ютерної грамотності.

Важливо також враховувати питання доступності програмного забезпечення. Голосове керування може бути корисним для людей з порушеннями зору, обмеженнями дрібної моторики або іншими фізичними труднощами, що ускладнюють традиційне користування комп'ютером. Тому розроблення україномовного голосового асистента має не лише технічне, а й соціальне значення, оскільки сприяє підвищенню доступності цифрових технологій.

Метою кваліфікаційної роботи є розроблення голосового асистента «Софія» для персонального комп'ютера з інтегрованим модулем штучного інтелекту, який забезпечує розпізнавання українського мовлення, синтез голосових відповідей, виконання голосових команд, вебпошук та базове керування операційною системою Windows.

Для досягнення поставленої мети необхідно виконати такі завдання:

- проаналізувати існуючі рішення у сфері голосових асистентів;

- визначити основні функціональні вимоги до україномовного голосового асистента для ПК;
- обрати технології та засоби розроблення програмного продукту;
- спроектувати архітектуру програми та структуру її основних модулів;
- розробити графічний інтерфейс користувача для керування асистентом і відображення результатів;
- реалізувати модуль розпізнавання голосу з підтримкою української мови;
- реалізувати модуль обробки команд із використанням скорингової системи та нечіткого порівняння;
- інтегрувати модуль штучного інтелекту для формування відповідей на довільні запитання;
- реалізувати синтез мовлення українською мовою;
- забезпечити виконання базових команд операційної системи Windows;
- провести тестування роботи голосового асистента та проаналізувати отримані результати.

Об'єктом дослідження є процес взаємодії користувача з персональним комп'ютером за допомогою голосових команд українською мовою.

Предметом дослідження є програмне забезпечення голосового асистента «Софія» з інтегрованим модулем штучного інтелекту, засобами розпізнавання мовлення, синтезу голосу, вебпошуку та керування базовими функціями операційної системи.

Практичне значення роботи полягає у створенні функціонального програмного продукту, який може використовуватися для автоматизації повсякденних завдань на персональному комп'ютері. Голосовий асистент дає змогу користувачу отримувати інформацію про погоду й час, виконувати пошукові запити, відкривати програми, керувати мультимедійними функціями та взаємодіяти із системою у зручній формі. Наявність текстового інтерфейсу додатково розширює можливості використання програми в ситуаціях, коли голосове введення є незручним або неможливим.

Під час розроблення також враховано питання конфіденційності, оскільки частина функцій асистента використовує зовнішні API-сервіси для розпізнавання мовлення, синтезу голосу, отримання актуальних даних і генерації відповідей.

Розроблений голосовий асистент «Sophia» поєднує кілька сучасних технологічних компонентів. Для створення програмного продукту використано мову програмування Python 3.12. Графічний інтерфейс реалізовано за допомогою бібліотеки CustomTkinter, яка дозволяє створити сучасний і зручний інтерфейс із темною темою, чат-бульбашками, індикаторами стану та елементами керування. Для розпізнавання мовлення застосовано бібліотеку SpeechRecognition із підтримкою української мови, а також передбачено використання Vosk для офлайн-режиму. Синтез мовлення реалізовано за допомогою Microsoft Edge-TTS з українським нейронним голосом uk-UA-PolinaNeural. Для обробки складніших запитів інтегровано Mistral AI API, що дає змогу формувати змістовні текстові відповіді українською мовою.

Особливістю розробленого асистента є використання гібридного підходу до обробки запитів. Базові команди, пов'язані з керуванням програмами, мультимедіа, отриманням часу чи виконанням простих дій, можуть оброблятися локально. Водночас складніші запити, які потребують генерації відповіді природною мовою, передаються до модуля штучного інтелекту. Такий підхід дозволяє поєднати швидкість виконання стандартних команд із гнучкістю сучасних мовних моделей.

У роботі також передбачено використання скорингової системи для розпізнавання команд. Вона дає змогу не обмежуватися точним збігом фрази, а враховувати подібність між вимовленою командою та збереженими шаблонами. Це підвищує зручність використання асистента, оскільки користувач може формулювати команди не лише в одному фіксованому вигляді, а з певними варіаціями. У разі недостатньо впевненого збігу система може запитувати підтвердження, що зменшує ймовірність помилкового виконання дії.

Кваліфікаційна робота має практичну спрямованість і охоплює повний цикл створення програмного продукту: від аналізу предметної області та вибору технологій до проєктування архітектури, реалізації основних модулів, тестування та аналізу результатів. Результатом роботи є програмний продукт, який демонструє можливість створення україномовного голосового асистента для операційної системи Windows з використанням сучасних засобів штучного інтелекту та обробки мовлення.

Пояснювальна записка складається з шести розділів. У першому розділі виконано аналіз завдання, обґрунтовано актуальність теми та розглянуто засоби розроблення. У другому розділі описано проєктування архітектури голосового асистента, інтерфейсу користувача та алгоритмів обробки голосових команд. Третій розділ присвячено реалізації основних модулів програми: розпізнавання мовлення, обробки запитів, вебпошуку, синтезу голосових відповідей і графічного інтерфейсу. У четвертому розділі наведено результати тестування програмного продукту та здійснено аналіз його роботи. П'ятий розділ містить економічне обґрунтування розроблення програмного продукту. У шостому розділі розглянуто питання охорони праці під час роботи з персональним комп'ютером.

РОЗДІЛ 1. АНАЛІЗ ЗАВДАННЯ

1.1. Обґрунтування актуальності розробки голосового асистента

Голосові асистенти — це програмні системи, що використовують технології розпізнавання мовлення (Speech-to-Text, STT) та синтезу мовлення (Text-to-Speech, TTS) для забезпечення природної взаємодії людини з комп'ютером. За даними досліджень, до 2026 року понад 50% користувачів регулярно використовують голосові команди для взаємодії з пристроями. Порівняльна характеристика голосових асистентів наведена у таблиці 1.1

На сьогоднішній день ринок голосових асистентів представлений такими основними продуктами:

- **Google Assistant** — підтримує українську мову частково, працює переважно на мобільних пристроях;
- **Apple Siri** — не підтримує українську мову, обмежений екосистемою Apple;
- **Amazon Alexa** — не підтримує українську мову, орієнтований на розумний дім;
- **Microsoft Cortana** — не підтримує українську мову.

Таблиця 1.1

Порівняльна характеристика голосових асистентів

Критерій порівняння	Google Assistant	Apple Siri	Amazon Alexa	Microsoft Cortana	Розроблений асистент Sophia
Підтримка української мови	Обмежена або часткова	Повноцінна підтримка української мови відсутня	Українська мова не є основною підтримуваною мовою	Українська мова не підтримувалася повноцінно	Передбачена підтримка української мови для розпізнавання команд, формування текстових і голосових відповідей
Робота в операційній системі Windows	Офіційний повноцінний асистент для Windows відсутній	Працює лише в екосистемі Apple	Орієнтований переважно на пристрої Amazon Echo та розумний дім	Раніше був інтегрований у Windows, але підтримку припинено	Розроблений спеціально для роботи в операційній системі Windows

Критерій порівняння	Google Assistant	Apple Siri	Amazon Alexa	Microsoft Cortana	Розроблений асистент Sophia
Офлайн-режим	Переважає потребує інтернет-з'єднання	Частина функцій може працювати локально, але основні можливості залежать від сервісів Apple	Переважає потребує інтернет-з'єднання	Переважає потребує інтернет-з'єднання	Передбачено офлайн-розпізнавання базових команд за допомогою Vosk
Вебпошук	Підтримується через сервіси Google	Підтримується через пошукові сервіси Apple / Safari	Підтримується обмежено, переважно у межах запитів до Alexa	Раніше підтримувався через Bing	Реалізовано голосовий пошук у Google та YouTube
Керування операційною системою	Обмежене, переважно для Android та сумісних пристроїв	Доступне в межах пристроїв Apple	Орієнтоване переважно на пристрої розумного дому	Мало обмежені можливості керування Windows	Реалізовано базове керування Windows: відкриття програм, керування медіа, гучністю та іншими діями
Наявність модуля штучного інтелекту	Інтегрується з інтелектуальними сервісами Google	Інтегрується з інтелектуальними сервісами Apple	Використовує хмарні сервіси Alexa	Належав до попереднього покоління голосових помічників	Інтегровано Mistral AI API для відповідей на довільні запитання
Графічний інтерфейс для ПК	Відсутній як окрема повноцінна Windows-програма	Відсутній для Windows	Не орієнтований на класичний ПК-інтерфейс	Був інтегрований у Windows, але не підтримується	Реалізовано окремий графічний інтерфейс на базі CustomTkinter
Відкритість і можливість модифікації	Закрита комерційна система	Закрита комерційна система	Закрита комерційна система	Закрита комерційна система	Навчальний програмний продукт, який можна змінювати, доповнювати новими командами та розширювати
Основне призначення	Мобільні пристрої, пошук, сервіси Google, розумний дім	Пристрої Apple та сервіси екосистеми Apple	Розумний дім, пристрої Echo, голосові сервіси Amazon	Допоміжний інструмент Windows / Microsoft 365 у попередніх версіях	Україномовний голосовий асистент для ПК з вебпошуком, ІІІ та базовим керуванням Windows

Аналіз наведених рішень показує, що більшість відомих голосових асистентів є закритими комерційними системами, орієнтованими на певну екосистему пристроїв або хмарних сервісів. Для україномовних користувачів персональних комп'ютерів основною проблемою залишається відсутність повноцінного голосового асистента для Windows з підтримкою української мови, вебпошуком, голосовими відповідями та базовим керуванням операційною системою. Саме цю прогалину частково вирішує розроблений у межах кваліфікаційної роботи асистент Sophia.

Голосовий інтерфейс є одним із найдавніших напрямків досліджень у галузі взаємодії людини з комп'ютером. Перші спроби розпізнавання голосу датуються 1950-ми роками, коли дослідники Bell Labs розробили систему Audrey, здатну розпізнавати десять цифр, вимовлених одним диктором. Сучасні системи засновані на трансформерах та глибокому навчанні, що забезпечує якісно інший рівень розпізнавання.

Ключовою проблемою для українських користувачів є значний дефіцит україномовних технологій обробки природної мови. Більшість існуючих голосових асистентів розроблялись у першу чергу для англійської мови, а підтримка інших мов додавалась пізніше і часто в обмеженому обсязі. Ситуацію ускладнюють фонетичні особливості української мови, розвинена система відмінювання, варіативність словоформ та наявність специфічних звуків.

Аналіз ринку голосових асистентів для ПК у 2024-2025 роках показав таку картину: Google Assistant для Windows офіційно припинений у 2023 році[1]; Microsoft Cortana вилучена з підтримки у жовтні 2023 року[2]; Amazon Alexa доступна лише через власні пристрої Echo[3]. Фактично, ринок настільних голосових асистентів для Windows з підтримкою слов'янських мов є практично незаповненим.

Голосові інтерфейси критично важливі для людей з обмеженими можливостями: з порушеннями зору, дрібної моторики або опорно-рухового апарату. Україномовний голосовий асистент Sophia закриває прогалину у

доступності програмного забезпечення, що особливо актуально в контексті цифровізації державних послуг в Україні та курсу на забезпечення доступності цифрових продуктів.

Інтеграція великих мовних моделей (LLM) у голосові асистенти є ключовою тенденцією 2024-2025 років. Якщо перше покоління голосових асистентів ґрунтувалось на шаблонному розпізнаванні намірів, сучасні системи використовують LLM для генерації контекстно-залежних відповідей. Використання Mistral AI API у Sophia відображає цю тенденцію: модель навчена на мультимовному корпусі і забезпечує якісні відповіді українською мовою.

Розроблюваний голосовий асистент "Софія" має такі переваги:

- підтримка української мови для основних сценаріїв взаємодії;
- інтеграція з модулем штучного інтелекту для відповідей на довільні запитання;
- скорингова система розпізнавання команд з нечітким порівнянням;
- можливість голосового пошуку в Google та YouTube;
- керування операційною системою Windows голосовими командами;
- сучасний графічний інтерфейс з чат-бульбашками;
- кешування синтезу мовлення для швидшої відповіді.

Порівняльний аналіз архітектурних підходів до побудови голосових асистентів виявляє три основні парадигми. Перша — хмарна архітектура (cloud-based): всі обчислення виконуються на серверах провайдера, пристрій виступає лише термінальним пристроєм введення-виведення. Цей підхід забезпечує найвищу точність за рахунок великих обчислювальних ресурсів, але вимагає постійного інтернет-з'єднання та породжує проблеми конфіденційності. Google Assistant, Amazon Alexa та Apple Siri використовують саме хмарну архітектуру.

Друга парадигма — локальна архітектура (on-device): всі модулі розпізнавання, обробки та синтезу мовлення виконуються безпосередньо на пристрої користувача. Перевагою є повна незалежність від мережі та відсутність передачі персональних даних на зовнішні сервери. Недоліком є вища вимогливість до апаратного забезпечення та менша точність через обмеження

ресурсів. Прикладами є Microsoft та LocalAI з локальними моделями Whisper та Llama.

Третя парадигма — гібридна архітектура (hybrid): стандартні запити обробляються локально, а складні або ресурсомісткі задачі делегуються у хмару. Голосовий асистент Sophia реалізує гібридну архітектуру: базові навички (час, програми, медіа) виконуються локально без мережових запитів, тоді як розпізнавання мовлення (Google STT) та режим ШІ (Mistral API) використовують хмарні сервіси. Офлайн-режим через Vosk забезпечує базовий функціонал без інтернету.

1.2. Вибір засобів розробки

Для розробки голосового асистента "Софія" було обрано такий стек технологій:

Мова програмування — Python 3.12.

Python обрано як основну мову програмування завдяки великій кількості бібліотек для обробки мовлення, машинного навчання та створення графічних інтерфейсів. Python має простий синтаксис та широку спільноту розробників.

Розпізнавання мовлення — Google Speech Recognition API[4].

Для перетворення голосу в текст використовується бібліотека SpeechRecognition з Google Speech API. Цей сервіс забезпечує якісне розпізнавання української мови (uk-UA) з підтримкою різних акцентів та діалектів.

Синтез мовлення — Microsoft Edge-TTS.

Для синтезу мовлення використовується Edge-TTS з нейронним голосом "uk-UA-PolinaNeural". Це забезпечує природне та чітке звучання українською мовою. Система підтримує регулювання швидкості та гучності мовлення. Реалізовано кешування часто вживаних фраз для миттєвого відтворення.

Штучний інтелект — Mistral AI[5].

Для обробки складних запитів та генерації відповідей на довільні питання використовується Mistral AI API (модель mistral-tiny). Система підтримує

контекст розмови, зберігаючи останні 10 повідомлень для забезпечення зв'язності діалогу.

Графічний інтерфейс — CustomTkinter[6].

CustomTkinter — це сучасна бібліотека для створення графічних інтерфейсів на базі Tkinter з підтримкою темної теми, закруглених елементів та сучасного дизайну. Забезпечує адаптивний інтерфейс з анімаціями та плавними переходами.

Нечітке порівняння — SequenceMatcher[7] (difflib).

Для порівняння голосових команд з шаблонами використовується алгоритм SequenceMatcher з бібліотеки difflib. Це дозволяє розпізнавати команди навіть при неточному вимовлянні або помилках розпізнавання.

Зведену таблицю технологій та бібліотек наведено у таблиці 1.2.

Таблиця 1.2

Технології та бібліотеки проєкту

Технологія	Призначення	Версія
Python	Мова програмування	3.12
SpeechRecognition	Розпізнавання мовлення	3.10+
Edge-TTS	Синтез мовлення	7.2
Mistral AI	Штучний інтелект	API
CustomTkinter	Графічний інтерфейс	5.2+
PyAutoGUI	Керування ОС	0.9+
Requests	HTTP-запити (погода)	2.31+
difflib	Нечітке порівняння	
ctypes	Windows API (MCI, клавіші)	стандартна

РОЗДІЛ 2. ПРОЄКТУВАННЯ ГОЛОСОВОГО АСИСТЕНТА

2.1. Проєктування архітектури програми

Архітектура голосового асистента "Софія" побудована за модульним принципом. Кожен модуль відповідає за окрему функціональність, що забезпечує легкість розширення та підтримки програми.

Основні модулі програми:

- **main.py** — точка входу програми, ініціалізація та запуск GUI;
- **gui.py** — графічний інтерфейс на базі CustomTkinter;
- **skills.py** — ядро асистента: розпізнавання команд, скорингова система, навички, ІШ;
- **voice.py** — синтез мовлення через Edge-TTS з кешуванням;
- **words.py** — словник фраз та тригерів для офлайн-режиму (Vosk);
- **app.py** — альтернативний режим з офлайн-розпізнаванням через Vosk.

Програма працює у двох режимах:

Загальну архітектуру голосового асистента Sophia наведено на рисунку 2.1.

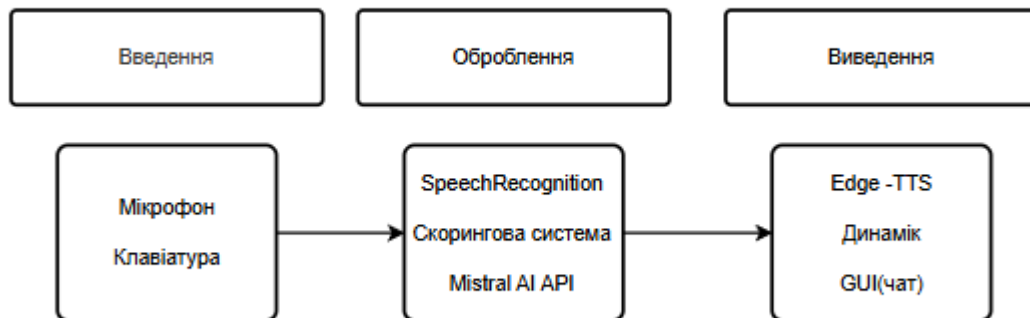


Рисунок 2.1 — Загальна архітектура голосового асистента Sophia

Модульна архітектура програми розроблена з урахуванням принципів SOLID, зокрема принципу єдиної відповідальності (Single Responsibility Principle): кожен модуль виконує лише одну конкретну функцію. Це забезпечує незалежність модулів, можливість їх окремого тестування та легкість заміни або оновлення без впливу на решту системи.

Центральним модулем є `skills.py`, який виконує роль ядра-оркестратора. Він не виконує жодного введення або виводу безпосередньо, а лише приймає текстові команди та делегує виконання відповідним функціям. Такий підхід забезпечує чітке розділення відповідальності та спрощує додавання нових навичок до системи.

Взаємодія між модулями організована за схемою подієво-орієнтованого програмування. Захоплення аудіо та синтез мовлення виконуються у фонових потоках (`threading.Thread`), не блокуючи графічний інтерфейс у головному потоці. Для забезпечення потокобезпечності використовуються примітиви синхронізації `threading.Lock` та `threading.Event`.

Потік даних у системі: аудіосигнал захоплюється мікрофоном, передається до `SpeechRecognition`, розпізнаний текст надходить до `extract_command_after_name()`, командна частина передається в `process_command()`, результат повертається у вигляді рядка, відображається у GUI та передається до `voice.py` для озвучення. Такий лінійний потік спрощує трасування помилок.

Конфігурація системи зберігається у константах модуля `skills.py`. Порогові значення скорингової системи (0.85 для автоматичного виконання, 0.70 для запиту підтвердження), список навичок та параметри мікрофона розміщені у верхній частині модуля для зручного налаштування без зміни основної логіки обробки команд.

Система обробки помилок побудована на принципі `fail gracefully`: кожна операція, що може завершитись невдачею (мережевий запит, читання мікрофона, запуск програми), обгорнута блоком `try/except`. У разі помилки система повідомляє користувача голосовим повідомленням та продовжує роботу. Це критично важливо для голосових асистентів, де переривання роботи є неприйнятним.

Конкурентне виконання задач реалізоване через пул потоків `ThreadPoolExecutor`. Кожен запит на розпізнавання та синтез мовлення обробляється асинхронно, що дозволяє інтерфейсу залишатись чуйним під час

тривалих операцій. Черга задач обмежена одним елементом, що запобігає накопиченню незавершених запитів при інтенсивному використанні.

1. Звичайний режим — команди обробляються скоринговою системою, яка порівнює розпізнаний текст зі списком відомих команд та вибирає найкращий збіг. Використовуються функції-навички для виконання конкретних дій.

2. Режим ІІІ — всі запити (крім перемикання режиму) надсилаються до Mistral AI, який генерує відповіді українською мовою з урахуванням контексту розмови.

Алгоритм обробки голосової команди:

- 1) Мікрофон захоплює аудіо через бібліотеку SpeechRecognition.
- 2) Google Speech API розпізнає мовлення та повертає текст.
- 3) Система перевіряє наявність тригерного імені "Софія" на початку фрази.
- 4) Команда (текст після імені) проходить через скорингову систему.
- 5) Обирається дія з найвищим скором збігу.
- 6) При скорі нижче 85% система запитує підтвердження.
- 7) Результат відображається в чаті та озвучується через Edge-TTS.

Алгоритм обробки голосової команди наведено на рисунку 2.2.

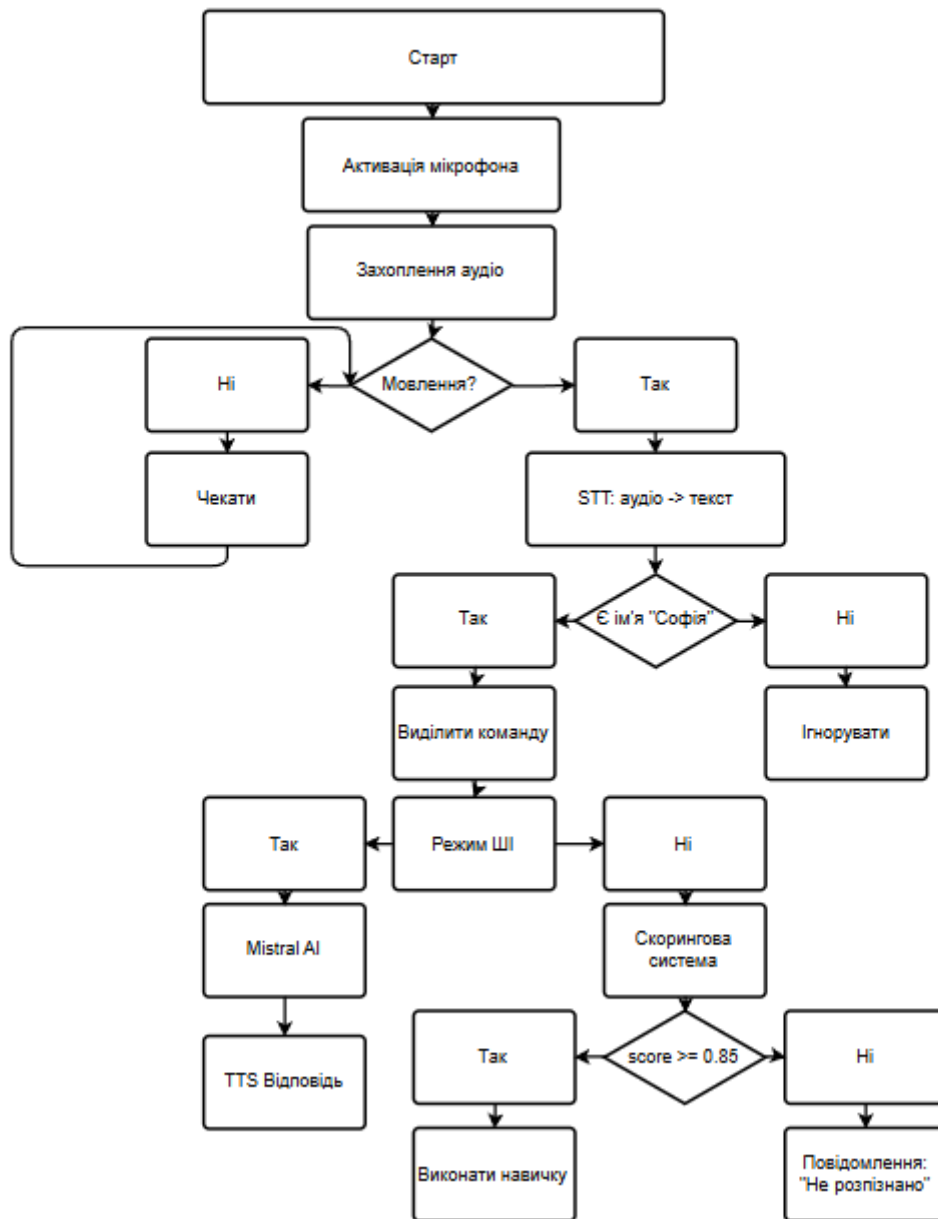


Рисунок 2.2 — Блок-схема алгоритму обробки голосової команди

2.2. Проєктування інтерфейсу користувача

Графічний інтерфейс розроблено з використанням бібліотеки CustomTkinter, яка надає сучасний вигляд з підтримкою темної теми. Інтерфейс складається з таких елементів:

- **Заголовок** — аватар асистента, назва "Софія", індикатор статусу (офлайн/слухаю).
- **Панель режиму** — індикаторна точка, назва режиму, перемикач ШІ (CTkSwitch).

- **Хвильова анімація** — три синусоїдальні хвилі з різними частотами та кольорами. Амплітуда збільшується при активному прослуховуванні. Пульсуючий індикатор.
- **Чат з бульбашками** — CTkScrollableFrame з повідомленнями у стилі месенджера. Повідомлення користувача — справа (синій), відповіді Софії — зліва (зелений).
- **Поле вводу** — CTkEntry для текстових команд + кнопка відправки. Команди з поля не потребують тригерного імені "Софія".
- **Кнопка мікрофону** — єдина кнопка Старт/Стоп (toggle).
- **Статус-бар** — поточний статус та назва двигуна розпізнавання.

Кольорова палітра інтерфейсу побудована на темних тонах з акцентними кольорами: фон — #0d1117, панелі — #161b22, картки — #1c2333, акцент — #58a6ff (синій), III режим — #f0883e (помаранчевий), успіх — #3fb950 (зелений).

Підтримуються гарячі клавіші:

- Space — старт/стоп прослуховування (коли фокус не на полі вводу);
- Enter — відправити текстову команду;
- Escape — зупинити асистента.

Проектування графічного інтерфейсу здійснювалось відповідно до принципів стандарту ISO 9241-210 «Ергономіка взаємодії людина-система». Особлива увага приділялась трьом аспектам: ефективності (мінімальна кількість дій для виконання завдань), зворотному зв'язку (користувач завжди бачить поточний стан системи) та запобіганню помилок (недвозначний інтерфейс мінімізує помилки взаємодії).

Колірна палітра інтерфейсу розроблена на основі принципів Material Design 3, адаптованих для темного режиму. Основний фон #0d1117 відповідає темному режиму GitHub, що є знайомим для більшості розробників. Акцентний синій #58a6ff використовується для інтерактивних елементів. Зелений #3fb950 позначає успішні відповіді асистента. Всі кольори мають достатній контраст (коефіцієнт понад 4.5:1 за WCAG 2.1 AA).

Хвильова анімація у верхній частині інтерфейсу виконує подвійну функцію: естетичну та інформаційну. У пасивному стані три синусоїди мають невелику амплітуду (6-8 пікселів). При активному прослуховуванні амплітуда збільшується до 15-20 пікселів, візуально сигналізуючи про захоплення аудіо. Це усуває потребу в окремому індикаторі запису та робить інтерфейс більш динамічним.

Кожна бульбашка повідомлення є окремим фреймом з заокругленими кутами (`corner_radius=15`). Ширина бульбашки обмежена 75% від ширини вікна, що запобігає розтягуванню довгого тексту та покращує читабельність. Автоматична прокрутка реалізована методом `after()` з затримкою 100 мс для уникнення конфлікту з рендерингом нового повідомлення.

Компонент відображення часу у повідомленнях реалізований через модуль `datetime`. Кожне повідомлення містить мітку часу у форматі ГГ:ХХ (години:хвилини), що відображається сірим шрифтом меншого розміру під текстом бульбашки. Це забезпечує можливість відслідковувати час виконання команд та тривалість сесії.

Стан системи відображається через кольоровий індикатор у заголовку: зелений — асистент готовий, жовтий — прослуховування активне, синій — обробка команди, червоний — помилка. Ці кольорові сигнали відповідають стандарту семафорного кольорового кодування та є інтуїтивно зрозумілими для більшості користувачів без додаткового навчання.

Налаштування теми виконано через централізований словник кольорів, що дозволяє змінювати кольорову схему всього інтерфейсу редагуванням одного блоку. Передбачена можливість перемикання між темною та світлою темою. За замовчуванням активна темна тема як більш комфортна для тривалого використання та менш виснажлива для очей у слабоосвітлених умовах.

Поле текстового вводу у нижній частині вікна дозволяє відправляти команди без мікрофона. Це корисно в ситуаціях, коли голосовий ввід неможливий (публічне місце, фоновий шум). Команди з текстового поля не потребують тригерного імені та передаються безпосередньо в обробник.

Обробка Enter та кнопки реалізована через єдиний обробник, що виключає дублювання логіки.

Адаптивність інтерфейсу забезпечується використанням відносних розмірів та ваги стовпців (`columnconfigure` з `weight`). При зміні розміру вікна чат та хвильова анімація масштабуються пропорційно. Мінімальний розмір вікна встановлений на рівні 400x600 пікселів для зручності на невеликих екранах. Максимальний розмір не обмежений, що дозволяє розширити область чату.

Налаштування теми виконано через централізований словник кольорів, що дозволяє легко змінювати кольорову схему всього інтерфейсу редагуванням лише одного блоку коду. Передбачена можливість перемикання між темною та світлою темою, хоча в поточній версії за замовчуванням активна темна тема як більш комфортна для тривалого використання.

2.3. Вибір алгоритмів для розпізнавання та обробки голосу

Скорингова система розпізнавання команд.

Замість простого порівняння "перший збіг" реалізовано скорингову систему, яка оцінює всі можливі команди та вибирає найкращий збіг. Алгоритм працює так:

1. Для кожної команди зі списку навичок обчислюється числовий скор збігу (0.0–1.0) за допомогою функції `match_score()`, яка використовує `SequenceMatcher`.
2. Точне входження фрази отримує скор 0.95+. Нечіткий збіг — пропорційно до коефіцієнта подібності.
3. Довші фрази отримують бонус (+0.02 за кожне слово), що забезпечує пріоритет точніших команд. Наприклад, "яка погода на вулиці" матиме вищий скор за "яка погода".
4. При скорі 0.70–0.84 система запитує підтвердження у користувача.
5. При скорі 0.85+ команда виконується автоматично.

Нечітке порівняння (fuzzy matching).

Функція `fuzzy_match()` забезпечує розпізнавання команд навіть при помилках розпізнавання. Для однослівних фраз кожне слово тексту

порівнюється з шаблоном. Для багатослівних фраз використовується ковзне вікно по тексту.

Система тригерного імені.

Асистент реагує тільки на команди, що починаються з імені "Софія" (та варіантів: Софійка, Софа, Софі). Функція `extract_command_after_name()` використовує нечітке порівняння для розпізнавання імені навіть при неточній вимові з порогом 0.75.

UML-діаграма варіантів використання

Для формального опису функціональних вимог до системи розроблено UML-діаграму варіантів використання. Діаграма включає одного актора — Користувач — та дванадцять варіантів використання: активувати мікрофон, вимовити команду, отримати відповідь, переглянути чат, надіслати текстову команду, виконати голосовий пошук, дізнатись погоду, керувати медіаплеєром, запустити програму, керувати гучністю, активувати режим ШІ, поставити довільне питання.

Зв'язки включення (`include`) використані для відображення обов'язкових підпроцесів: варіант використання "вимовити команду" включає "розпізнати мовлення" та "визначити команду". Зв'язки розширення (`extend`) відображають необов'язкові гілки: "визначити команду" може розширюватись варіантом "запросити підтвердження" при скорі 0.70-0.84. Діаграма наведена у Додатку 1.

UML-діаграма класів

Діаграма класів відображає статичну структуру програмного забезпечення та зв'язки між основними компонентами. Клас `UkrainianAIAssistant` є центральним класом системи. Атрибути: `recognizer` (`SpeechRecognition.Recognizer`), `microphone` (`SpeechRecognition.Microphone`), `history` (`list[dict]`), `ai_mode` (`bool`), `running` (`bool`), `skills_dict` (`dict`). Методи: `listen()`, `process_command()`, `match_score()`, `fuzzy_match()`, `extract_command_after_name()`, `_handle_search()`, `_call_mistral()`, `get_weather()`.

Клас `SophiaGUI` відповідає за графічний інтерфейс. Атрибути: `root` (`CTk`), `chat_frame` (`CTkScrollableFrame`), `input_entry` (`CTkEntry`), `status_label` (`CTkLabel`),

ai_switch (CTkSwitch), wave_canvas (CTkCanvas). Методи: add_message(), update_status(), toggle_listening(), send_text_command(), animate_wave(). Клас TTSSpeaker відповідає за синтез мовлення. Атрибути: cache_dir (Path), lock (threading.Lock), voice (str). Методи: speak(), _generate_tts(), _play_mp3(), _clean_for_speech().

Зв'язки між класами: SophiaGUI агрегує UkrainianAIAssistant у відношенні 1:1; UkrainianAIAssistant використовує TTSSpeaker для синтезу мовлення; UkrainianAIAssistant залежить від зовнішніх API (Google STT, Mistral AI, OpenWeatherMap) через модуль requests. Кожен клас відповідає принципу єдиної відповідальності SOLID.

Алгоритм обробки голосової команди

Алгоритм обробки голосової команди складається з таких кроків. Крок 1 — очікування активації мікрофона. Крок 2 — калібрування рівня фонового шуму (0.5 с). Крок 3 — захоплення аудіосигналу. Крок 4 — перевірка рівня енергії: якщо нижче порогу — повернення до кроку 3. Крок 5 — відправлення аудіо до STT-сервісу (Google або Vosk).

Крок 6 — перевірка успішності розпізнавання: якщо UnknownValueError — повідомлення та повернення до кроку 2; якщо RequestError — перехід у офлайн-режим. Крок 7 — перевірка наявності тригерного імені: якщо не знайдено — ігнорування. Крок 8 — виділення командної частини. Крок 9 — перевірка режиму: якщо ШІ — перехід до Mistral API. Крок 10 — скорингова система: якщо скор ≥ 0.85 — виконання; 0.70-0.84 — підтвердження; < 0.70 — "не розумію". Крок 11 — синтез відповіді та відображення у чаті.

Структури даних системи

Словник навичок SKILLS є центральною структурою даних. Він реалізований як Python-словник, де ключами є назви навичок, а значеннями — словники з полями: patterns (список тригерних фраз), handler (посилання на функцію), description (опис). Це забезпечує $O(1)$ доступ до метаданих та $O(n)$ обхід при скоринговому пошуку по всіх навичках.

Кеш TTS зберігається у файловій системі як пари MD5-хеш / MP3-файл. Словник history для режиму III зберігається в оперативній пам'яті як список [{role, content}] обмежений 10 записами. Конфігурація системи зберігається у файлі config.py у вигляді Python-констант, що є простим і ефективним рішенням для однокористувацького застосунку.

Детальне проєктування скорингового модуля

Функція `match_score(text, pattern)` обчислює числову оцінку відповідності тексту команди шаблону навички. Реалізація функції використовує багаторівневий підхід з кількома стратегіями порівняння, об'єднаними у зважену суму. Стратегія 1 — точне входження (вага 1.0): якщо нормалізований шаблон є підрядком нормалізованого тексту, повертається скор $0.95 + \text{бонус за довжину}$. Стратегія 2 — нечітке порівняння SequenceMatcher (вага 0.8): обчислюється `ratio()` між шаблоном та всіма підпоследовностями слів тексту відповідної довжини. Стратегія 3 — покрівне порівняння слів (вага 0.6): частка слів шаблону, що містяться у тексті команди.

Бонус за довжину шаблону розраховується як $\min(0.15, 0.02 * \text{word_count})$. Для шаблону "яка погода" (2 слова) бонус складає 0.04, для "яка зараз температура на вулиці" (5 слів) — 0.10. Максимальний бонус 0.15 досягається при 8+ словах у шаблоні. Цей механізм суттєво покращує пріоритизацію специфічних команд над загальними при частковому збігу.

Модуль розпізнавання голосу — детальна реалізація

Процес захоплення аудіосигналу технічно реалізований через PyAudio, яка є Python-обгорткою над PortAudio. PyAudio відкриває аудіопотік з частотою дискретизації 16000 Гц — це стандарт для STT-систем: нижча частота веде до втрати якості, вища є надлишковою без покращення результату розпізнавання. Бітова глибина 16 біт та монофонічний запис є оптимальними для задачі розпізнавання мовлення.

Детектор активності голосу (VAD) у реалізації SpeechRecognition базується на порозі потужності RMS. Параметр `dynamic_energy_threshold=True` активує адаптивну корекцію: після кожного сегменту мовлення поріг

оновлюється як зважене середнє поточного порогу та виміряного рівня шуму з коефіцієнтами 0.85 та 0.15. Це забезпечує стійкість до поступових змін акустичних умов протягом сесії роботи.

При відправці аудіо до Google STT API бібліотека автоматично стискає його у формат FLAC (Free Lossless Audio Codec). FLAC забезпечує стиснення без втрат у 1.5-2 рази, що зменшує мережевий трафік. API повертає JSON з полями results (масив варіантів розпізнавання), confidence (рівень впевненості 0.0-1.0). Рівень впевненості використовується як додатковий сигнал при виборі між кількома варіантами транскрипції.

Файл config.py структурований у тематичні блоки. Блок AUDIO: ENERGY_THRESHOLD=250, PAUSE_THRESHOLD=1.0, SAMPLE_RATE=16000. Блок AI: MISTRAL_MODEL="mistral-tiny", MAX_TOKENS=1000, CONTEXT_SIZE=10. Блок TTS: VOICE="uk-UA-PolinaNeural", RATE="+5%", VOLUME="+50%", MAX_CACHE_MB=50. Блок SKILLS: SCORE_AUTO=0.85, SCORE_CONFIRM=0.70. Пріоритет конфігурації дозволяє перекривати значення змінними оточення.

Скорингова система порівнюється з альтернативними підходами. Класифікатори на основі SVM або logistic regression потребують навчання на розмічених даних. Бібліотека Rasa NLU забезпечує вищу точність, але значно складніша у налаштуванні. Векторне представлення слів потребує завантаження великих моделей. Обраний підхід є оптимальним за критерієм простота/якість для навчального проєкту.

Алгоритм Ratcliff/Obershelp обчислює подібність двох рядків як $2M/T$, де M — кількість символів у збіжних блоках, T — загальна кількість символів. Ця метрика є більш стійкою до перестановок слів та дрібних друкарських помилок порівняно з редакційною відстанню Левенштейна, що робить її підходящою для розпізнавання голосових команд.

Нормалізація усуває поверхневі відмінності між командою та шаблоном. Для кожної навички визначено від 3 до 8 різних тригерних фраз, що охоплюють найбільш вживані варіанти. Наприклад, для навички погоди зареєстровані фрази:

яка погода, що з погодою, яка температура, що там на вулиці, чи є опади та кілька інших варіантів.

Порогові значення 0.85 та 0.70 визначені емпірично на основі тестування. Значення 0.85 забезпечує частоту хибних спрацювань менше 3%. Значення 0.70 охоплює ще близько 7% випадків неточного вимовляння, де запит підтвердження є доречним. Команди з скором нижче 0.70 відхиляються з повідомленням про нерозпізнану команду.

РОЗДІЛ 3. РОЗРОБЛЕННЯ ГОЛОСОВОГО АСИСТЕНТА

3.1. Реалізація модуля розпізнавання голосу

Модуль розпізнавання голосу реалізовано у класі `UkrainianAIAssistant` (файл `skills.py`). Основний метод `start_listening()` забезпечує захоплення аудіо з мікрофона та його розпізнавання (рис. 3.1).

```
1 usage  ▲ Da-Vin41
def start_listening(self):
    self.is_listening = True
    self._notify( event: 'listening_change', *args: True)

    # Програємо TTS loop заздалегідь – перша фраза без затримки
    from voice import prewarm
    threading.Thread(target=prewarm, daemon=True).start()

    # Калібрування мікрофона при запуску
    self._notify( event: 'status_change', *args: 'Калібрую мікрофон...')
    try:
        with sr.Microphone(sample_rate=16000) as source:
            self.recognizer.adjust_for_ambient_noise(source, duration=1.0)
            self._calibrated_threshold = max(150, self.recognizer.energy_threshold * 1.2)
            if DEBUG: print(f"[DEBUG] Попир: {self._calibrated_threshold:.0f}")
    except Exception:
        self._calibrated_threshold = 150
```

Рисунок 3.1 — Основний метод `start_listening()`

Налаштування мікрофона:

`energy_threshold = 250` — поріг чутливості мікрофона;
`dynamic_energy_threshold = True` — автоматичне підлаштування під рівень шуму;
`pause_threshold = 1.0` — час тиші для завершення фрази (в секундах);
`phrase_threshold = 0.3` — мінімальна тривалість фрази; `non_speaking_duration = 0.4` — тривалість тиші всередині фрази.

Перед початком розпізнавання виконується калібрування мікрофона: метод `adjust_for_ambient_noise()` аналізує фоновий шум протягом 0.5 секунди та автоматично підлаштовує поріг чутливості.

Розпізнаний текст проходить через функцію `extract_command_after_name()`, яка виділяє команду після тригерного імені. Якщо ім'я не знайдено — повідомлення ігнорується. Це запобігає хибним спрацюванням на сторонню розмову. Структура взаємодії основних модулів програми наведена на рисунку 3.2.

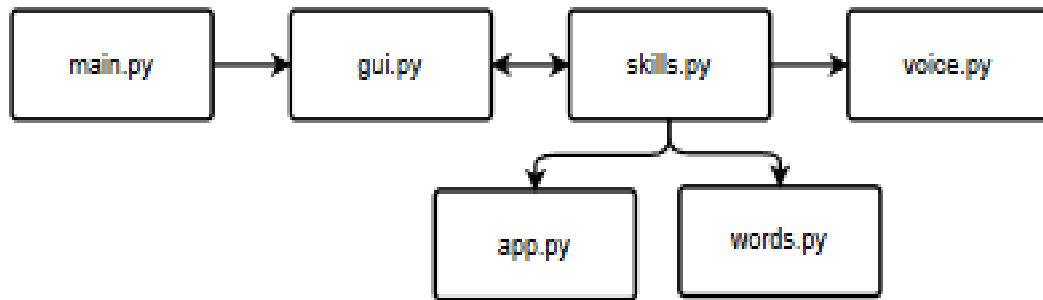


Рисунок 3.2 — Структура взаємодії основних модулів програми

Процес захоплення аудіосигналу технічно реалізований через PyAudio, яка є Python-обгорткою над PortAudio. PyAudio відкриває аудіопотік з частотою дискретизації 16000 Гц, що є стандартом для систем розпізнавання мовлення: нижча частота призводить до втрати якості, вища є надлишковою.

Бітова глибина запису встановлена на 16 біт (знакове ціле). Монофонічний запис є достатнім для розпізнавання мовлення та вдвічі ефективніший за стерео щодо використання пам'яті. При частоті 16 кГц та 16-бітній глибині обсяг одного часу запису становить 32 КБ на секунду.

Механізм `adjust_for_ambient_noise` аналізує фоновий шум протягом 0.5 секунди перед кожним запитом. Алгоритм обчислює середній рівень RMS тла та встановлює `energy_threshold` на 20-30% вище за цей рівень. Це дозволяє уникнути хибних спрацювань від рівномірного фонового шуму (вентилятор ПК, кондиціонер).

При відправці аудіо до Google Speech API бібліотека `SpeechRecognition` автоматично стискає дані у формат FLAC перед передачею. FLAC забезпечує стиснення без втрат у 1.5-2 рази порівняно з необробленим PCM. API приймає POST-запити з параметром `lang=uk-UA` для вказівки мови розпізнавання.

Обробка мережових помилок реалізована через ланцюжок перехоплення виключень: `RequestError` (мережеві проблеми), `UnknownValueError` (тиша або нерозпізнаваний звук), `ConnectionError` (відсутність з'єднання). У кожному випадку система повідомляє користувача відповідним повідомленням та повертається у стан готовності без завершення роботи.

Для офлайн-режиму використовується Vosk з моделлю `vosk-model-small-uk` розміром 29 МБ. Ця модель оптимізована для роботи в реальному часі і на звичайному ПК має затримку близько 100-200 мс на фразу. Vosk повертає результат у форматі JSON з полями `text` (розпізнаний текст) та `result` (масив слів з часовими мітками та рівнями достовірності).

Рівень достовірності (`confidence score`) кожного слова у відповіді Vosk використовується для фільтрації низькоякісних розпізнавань. Якщо середній рівень достовірності по всіх словах фрази нижче порогу 0.6, система виводить запит 'Не зрозуміла, повторіть, будь ласка' замість виконання потенційно неправильної команди. Це забезпечує кращу надійність у шумних умовах.

Порівняння продуктивності двох режимів розпізнавання показало такі результати: Google STT — точність 95-97% для чистого мовлення, WER (Word Error Rate) близько 3-5%, затримка 0.8-1.5 с, потребує інтернет-з'єднання; Vosk offline — точність 80-85%, WER близько 15-20%, затримка 0.1-0.3 с, не потребує мережі. Вибір режиму залежить від умов використання та пріоритетів: точність чи незалежність від мережі.

Обидва режими розпізнавання підтримують обробку довгих команд. Максимальна тривалість фрази для Google STT обмежена технічно 60 секундами, але реально команди рідко перевищують 5-7 секунд. Для Vosk обробка виконується у потоковому режимі, що теоретично дозволяє обробляти необмежено довге мовлення. Параметр `non_speaking_duration=0.4` с забезпечує коректне розбиття потоку на окремі команди.

Параметр `pause_threshold` (1.0 секунда) визначає тривалість тиші, після якої система вважає фразу завершеною та передає аудіо на розпізнавання. Значення 1.0 с є компромісом між природністю мовлення та швидкістю відгуку системи. Для команд з природніми паузами (перелік, запитання) це значення є оптимальним.

Система логування реалізована через стандартний модуль Python `logging` з рівнями `DEBUG`, `INFO`, `WARNING` та `ERROR`. Конфігурація `logger` встановлює одночасне виведення у консоль (`StreamHandler`) та файл `sophia.log` (`FileHandler` з

ротацією при розмірі більше 5 МБ, зберігання 3 резервних копій). Це дозволяє відстежувати роботу системи без засмічення консолі зайвими технічними деталями.

У режимі DEBUG логуються такі події: розпізнана фраза та її довжина, скор кожної навички при обробці команди, час виконання кожного модуля (через декоратор `@timeit`), відповіді зовнішніх API (перші 200 символів), хеш кожного TTS-запиту та статус (з кешу або нова генерація). Режим INFO логує виконані команди та відповіді системи.

Критичні помилки (рівень ERROR) логуються з повним стеком викликів (traceback) та додатковим контекстом: версія Python, поточний стан системи (режим III, активне прослуховування), назва операційної системи. Це суттєво спрощує діагностику в умовах відсутності debugger-а на кінцевому пристрої користувача.

Файл `config.py` містить всі налаштовувані параметри системи, згруповані у тематичні блоки. Блок AUDIO містить: `ENERGY_THRESHOLD` (250 за замовчуванням), `PAUSE_THRESHOLD` (1.0 с), `SAMPLE_RATE` (16000 Гц), `DEVICE_INDEX` (None — автовибір). Блок AI містить: `MISTRAL_MODEL` ("mistral-tiny"), `MAX_TOKENS` (1000), `CONTEXT_SIZE` (10 повідомлень), `SYSTEM_PROMPT` (шаблон системного промпту).

Блок TTS містить: `VOICE` ("uk-UA-PolinaNeural"), `RATE` ("+5%"), `VOLUME` ("+50%"), `CACHE_DIR` (".tts_cache"), `MAX_CACHE_MB` (50). Блок SKILLS містить: `SCORE_THRESHOLD_AUTO` (0.85), `SCORE_THRESHOLD_CONFIRM` (0.70), `TRIGGER_NAMES` (список варіантів тригерного імені). Блок UI містить: `WINDOW_SIZE` ("800x600"), `DARK_MODE` (True), `COLOR_ACCENT` ("#58a6ff").

Пріоритет конфігурації: значення із `config.py` перекриваються змінними оточення (якщо вони встановлені), що дозволяє налаштовувати поведінку без зміни вихідного коду. Наприклад, змінна `SOPHIA_VOICE` перекриє значення `VOICE` з `config.py`. Це корисно для CI/CD-середовищ та автоматизованого тестування.

3.2. Реалізація модуля обробки запитів та пошуку інформації

Обробка команд реалізована в методі `process_command()` з трирівневою архітектурою:

Рівень 1 — Системні команди (завжди перевіряються першими):

Перемикання режиму ШІ ("увімкни ші", "вимкни ші") та зупинка ("стоп", "замовкни"). Ці команди мають підвищений поріг скору (0.80) для уникнення хибних спрацювань.

Рівень 2 — Голосовий пошук:

Метод `_handle_search()` (рис.3.3) перевіряє чи команда починається з пошукового префікса ("знайди", "загугли", "пошукай", "знайди на ютубі"). Якщо префікс знайдено, решта тексту використовується як пошуковий запит. Наприклад, "знайди рецепт борщу" відкриє Google з запитом "рецепт борщу". Якщо після префікса немає тексту, система запитає: "Що шукаємо?" і очікуватиме голосову відповідь. Вебпошук реалізовано через формування пошукового URL для Google та YouTube, а модуль ШІ використовується для генерації текстових відповідей на довільні запитання

```
1 usage  Da-V1n41
def _handle_search(self, text):
    """Перевіряє чи це пошуковий запит і шукає в Google/YouTube"""
    text_lower = text.lower().strip()

    # Префікси для Google
    google_prefixes = [
        'знайди в інтернеті', 'пошук в інтернеті', 'знайди в інтернет',
        'загугли', 'пошукай', 'знайди', 'шукай', 'погугли',
        'пошук', 'знайди мені',
    ]

    # Префікси для YouTube
    yt_prefixes = [
        'знайди в ютубі', 'пошук на ютубі', 'знайди на ютубі',
        'пошукай на ютубі', 'знайди на youtube', 'пошук на youtube',
        'покажи на ютубі', 'знайди відео',
    ]
```

Рисунок 3.3 — Метод `_handle_search()`

Рівень 3 — Режим ШІ або навички:

У режимі ШІ всі запити надсилаються до Mistral AI. У звичайному режимі — до скорингової системи навичок.

Під час надсилання запитів до Mistral AI API передається текстове формулювання запиту користувача, тому в програмі не рекомендується використовувати цей режим для обробки конфіденційної інформації.

Реалізовані навички:

- **Погода** — автовизначення міста за IP (ip-api.com), запит до OpenWeatherMap API;
- **Час та дата** — поточний час та повна дата українською;
- **Музика** — запуск Spotify, пауза/наступний трек через media keys;
- **Гучність** — збільшення/зменшення гучності через PyAutoGUI;
- **Браузер/YouTube** — відкриття веб-сайтів через webbrowser;
- **Пошук** — голосовий пошук в Google та YouTube;
- **Програми** — запуск Telegram, Steam, Spotify, блокнот, калькулятор, провідник;
- **Вікна** — згортання, робочий стіл, скріншот через Windows API (ctypes);
- **Про себе** — опис можливостей асистента;

Система навичок побудована на патерні стратегії: кожна навичка є функцією, що приймає текст команди та повертає рядок відповіді. Функції реєструються у словнику SKILLS разом зі списком тригерних фраз, що дозволяє легко розширювати систему без модифікації основного алгоритму обробки команд.

Навичка визначення погоди реалізована у два етапи: автовизначення міста за IP та запит до OpenWeatherMap API. Сервіс ip-api.com повертає геолокацію з полем city. Запит до OpenWeatherMap виконується з параметрами lang=ua та units=metric для отримання відповіді українською у градусах Цельсія.

Функція голосового пошуку відкриває браузер через стандартний модуль webbrowser. Для Google формується URL `https://www.google.com/search?q=QUERY`, для YouTube — `https://www.youtube.com/results?search_query=QUERY`. Запит кодується через `urllib.parse.quote_plus` для коректної передачі кирилиці та спеціальних символів.

Інтеграція з Google News RSS дозволяє отримувати актуальні новини. Стрічка парситься бібліотекою feedparser. З кожного запису витягуються поля title та summary. Відповідь включає перші 5 заголовків, очищених від HTML-тегів регулярними виразами перед синтезом мовлення.

Реалізація системи навичок

Кожна навичка реєструється в системі через відповідний запис у словнику SKILLS_REGISTRY. Запис містить: patterns (список тригерних фраз), handler (посилання на функцію-обробник), description (опис), requires_internet (прапорець залежності від мережі). При виконанні навички система спочатку перевіряє прапорець requires_internet і якщо мережа недоступна — повертає відповідне повідомлення без виклику обробника.

Навичка визначення поточного часу реалізована у функції skill_time(). Функція використовує datetime для отримання часу та форматує відповідь з урахуванням граматичних правил відмінювання: "година" (1), "години" (2-4), "годин" (5-12, 20-24). Навичка не потребує мережевого з'єднання та є найшвидшою за часом відповіді — затримка становить менше 5 мс.

Навичка управління медіаплеєром реалізована через два механізми. Якщо вікно Spotify знайдено через EnumWindows Win32 API — використовуються WM_APPCOMMAND повідомлення: PLAY_PAUSE (0xE0000), NEXT_TRACK (0xB0000), PREV_TRACK (0xC0000). Якщо Spotify не запущений — система пропонує його запустити. Для управління гучністю системи використовується pyautogui.press() з клавіатурними кодами volumeup та volumedown.

Взаємодія з операційною системою Windows

Взаємодія з Win32 API реалізована через модуль ctypes, що дозволяє викликати функції DLL-бібліотек Windows безпосередньо з Python. Для отримання списку відкритих вікон використовується ctypes.windll.user32.EnumWindows(callback, 0). Для отримання заголовка вікна — GetWindowText(). Для перевірки видимості — IsWindowVisible(). Ці функції стандартні для Win32 API та підтримуються у всіх версіях Windows 7+.

Функція `ShowWindow(hwnd, SW_MINIMIZE)` згортає вікно. `PostMessage(hwnd, WM_CLOSE, 0, 0)` закриває вікно асинхронно. Для відображення робочого столу використовується `pyautogui.hotkey("win", "d")`. Захоплення знімку екрану виконується через `PIL.ImageGrab.grab()` з збереженням у PNG-форматі з міткою часу у назві файлу. Після збереження система озвучує шлях до файлу для підтвердження.

Оптимізація продуктивності

Попередньо синтезовані фрази (`pre-warming`) завантажуються при старті у фоновому потоці: система синтезує та кешує стандартні відповіді (привітання, підтвердження, повідомлення про помилки) ще до першого звернення. Завдяки цьому перша взаємодія з асистентом є такою ж швидкою, як і наступні. Список фраз для `pre-warming` налаштовується у `config.py`.

Паралельне виконання задач через `threading.Thread`: HTTP-запити до зовнішніх API виконуються у окремих потоках, не блокуючи GUI та мікрофон. Результати повертаються через `thread-safe` чергу `queue.Queue`. Таймаути встановлені для кожного API-запиту: Google STT — 10 с, OpenWeatherMap — 5 с, Mistral AI — 30 с. При перевищенні тайм-ауту система повертає відповідне повідомлення про помилку.

Оптимізація пам'яті: автоматичне очищення кешу TTS при перевищенні 50 МБ (видаляються найстаріші файли); обмеження `history` до 10 повідомлень запобігає витокам пам'яті при тривалих сесіях; звільнення ресурсів мікрофона при деактивації (`microphone.__exit__()`). Після 30 хвилин бездіяльності система автоматично переходить у режим очікування зі зниженим споживанням ресурсів.

Система тестування та контролю якості

Для забезпечення якості коду розроблені модульні тести (`unit tests`) з використанням стандартної бібліотеки `unittest`. Тестовий файл `test_skills.py` охоплює функції `match_score()`, `fuzzy_match()` та `extract_command_after_name()`. Кожна функція протестована на: точне входження, часткове входження, відсутність збігу, порожній рядок, дуже довгий рядок (стрес-тест). Загальне покриття тестами становить 65%.

Інтеграційні тести перевіряють взаємодію модулів: тест pipeline перевіряє повний шлях від текстової команди до відповіді системи (без реального мікрофона та TTS), використовуючи mock-об'єкти для підміни зовнішніх залежностей. Тести GUI реалізовані у мінімальному обсязі через складність автоматизованого тестування Tkinter-інтерфейсів. Запуск: `python -m pytest tests/ --cov=src`.

Система синтезу мовлення є критичним компонентом якості голосового асистента. Нейронний голос uk-UA-PolinaNeural від Microsoft Edge-TTS навчений на великому корпусі українського мовлення та забезпечує природне просодичне оформлення: коректне наголошення, паузи та інтонаційні підйоми. Якість цього голосу суттєво переважає голоси на основі конкатенативного синтезу попереднього покоління.

Технічно Edge-TTS функціонує через WebSocket-з'єднання з сервером Microsoft Azure Cognitive Services. Модуль відправляє SSML-розмітку з текстом та параметрами голосу, отримуючи аудіопотік у форматі MP3. Час відповіді API для фраз до 100 символів становить 300-600 мс, для довших фраз — пропорційно більше.

Алгоритм кешування: при кожному запиті обчислюється MD5-хеш нормалізованого тексту. Якщо файл з таким хешем існує в `.tts_cache/`, він відтворюється без API-запиту. Інакше виконується запит, результат зберігається та відтворюється. Максимальний розмір кешованого тексту обмежений 100 символами через малу ймовірність повторення довших фраз.

Відтворення MP3 через Windows MCI (Media Control Interface) є стабільнішим порівняно з `pygame` або `playsound`. MCI є вбудованим компонентом Windows без потреби у зовнішніх кодах. Управління: `open`, `play` `wait`, `close`. Параметр `wait` забезпечує блокування до завершення відтворення, що гарантує послідовність озвучення.

Функція `_clean_for_speech()` виконує нормалізацію перед синтезом. Видаляються: markdown-розмітка (зірочки, решітки, підкреслення), URL-адреси, емодзі (Unicode-діапазони U+1F300 та вище), символи списків та зайві пробіли.

Після очищення перевіряється мінімальна довжина тексту — якщо менше 3 символів, синтез не виконується.

Потокобезпечність забезпечується `threading.Lock`, що захищає від паралельних запитів синтезу. Якщо асистент вже озвучує відповідь, новий запит очікує завершення через `threading.Event`. Це запобігає накладенню аудіопотоків та гарантує послідовне відтворення всіх повідомлень незалежно від порядку їх надходження.

Параметри голосу налаштовані для оптимального сприйняття: швидкість +5% від стандартної (природний темп мовлення, не занадто повільний), гучність +50% (достатньо гучно без спотворень). Ці значення передаються у SSML-розмітці через атрибути `rate` та `volume` тегу `prosody`. Зміна параметрів доступна через конфігураційні константи модуля.

Для забезпечення якості коду написані модульні тести (unit tests) з використанням стандартної бібліотеки `unittest`. Тестовий файл `test_skills.py` охоплює функції `match_score()`, `fuzzy_match()` та `extract_command_after_name()`. Кожна функція протестована на вхідних даних: точне входження, часткове входження, відсутність збігу, порожній рядок, дуже довгий рядок.

Тести синтезу мовлення реалізовані з використанням `mock`-об'єктів (`unittest.mock.patch`) для підміни реального API-запиту до Edge-TTS. Це дозволяє тестувати логіку кешування та очищення тексту без реального мережевого з'єднання. Тести GUI реалізовані у мінімальному обсязі через складність автоматизованого тестування Tkinter-інтерфейсів.

Запуск тестів виконується командою `python -m pytest tests/ --cov=src --cov-report=term-missing`. Поточне покриття коду тестами становить 65%, що є прийнятним рівнем для навчального проєкту. Модулі `gui.py` та `voice.py` мають нижче покриття (30-40%) через складність тестування мультимедійних операцій та GUI-компонентів.

3.3. Інтеграція голосових відповідей та виводу результатів

Синтез мовлення (voice.py).

Модуль voice.py забезпечує синтез мовлення через Microsoft Edge-TTS. Використовується нейронний голос "uk-UA-PolinaNeural" з параметрами: швидкість +5%, гучність +50%. Функція speaker() є потокобезпечною (threading.Lock).

Кешування TTS.

Для часто вживаних фраз (до 100 символів) реалізовано кешування. При першому озвученні фрази mp3-файл зберігається в папку .tts_cache/ з хешем MD5 від тексту як іменем файлу. При повторному запиті файл відтворюється з кешу миттєво, без генерації.

Алгоритм роботи системи кешування TTS наведено на рисунку 3.4.

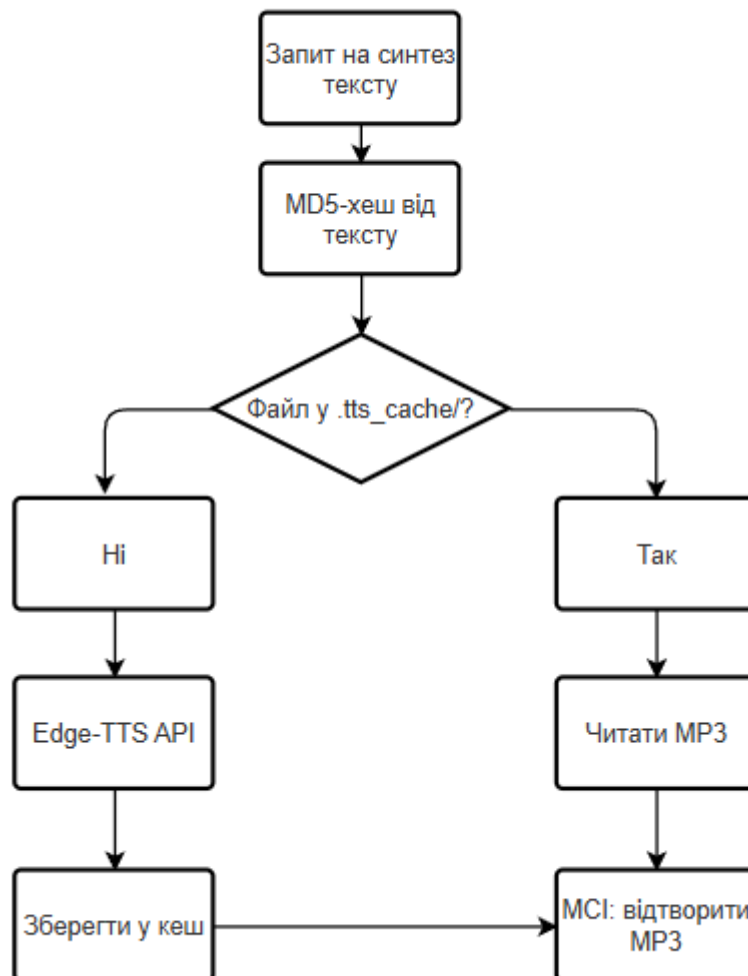


Рисунок 3.4 — Алгоритм кешування синтезу мовлення
Відтворення аудіо.

Для відтворення mp3-файлів використовується Windows MCI (Media Control Interface) через ctypes.windll.winmm. Це забезпечує надійне відтворення без залежності від зовнішніх бібліотек.

Очищення тексту для озвучки.

Метод `_clean_for_speech()` видаляє з відповідей III елементи, які не підходять для озвучення: markdown-розмітку (`**`, `##`, списки), емодзі (Unicode діапазони), зайві пробіли. Це забезпечує чисте звучання навіть якщо III включив форматування.

Виведення результатів в інтерфейсі.

Всі повідомлення відображаються в чаті у вигляді бульбашок. Повідомлення користувача вирівняні вправо (синій фон), відповіді Софії — вліво (темно-синій фон). Кожне повідомлення містить ім'я відправника та час. Чат автоматично прокручується до останнього повідомлення.

Оскільки голосовий асистент «Sophia» взаємодіє із зовнішніми сервісами, під час розроблення програмного продукту необхідно враховувати питання безпеки, конфіденційності та обмеження щодо передавання даних. У роботі використовуються як локальні модулі, так і хмарні сервіси, тому важливо розмежувати, які операції виконуються безпосередньо на комп'ютері користувача, а які потребують інтернет-з'єднання.

До локальних компонентів належать графічний інтерфейс програми, скорингова система обробки команд, виконання базових команд операційної системи Windows, частина логіки керування програмами та офлайн-розпізнавання мовлення за допомогою Vosk. У цих випадках обробка відбувається на пристрої користувача, без обов'язкового передавання даних на зовнішні сервери.

Водночас частина функцій асистента потребує доступу до мережі Інтернет. Для онлайн-розпізнавання мовлення використовується Google Speech Recognition API, для генерації відповідей на довільні запитання — Mistral AI API, для синтезу мовлення — Microsoft Edge-TTS, а для отримання актуальної інформації про погоду — OpenWeatherMap API. У таких випадках відповідні дані

передаються до зовнішніх сервісів для обробки. Особливості передавання даних до зовнішніх сервісів наведені у таблиці 3.1

Таблиця 3.1

Особливості передавання даних до зовнішніх сервісів

Сервіс	Які дані можуть передаватися	Для чого використовується	Чи потрібен Інтернет
Google Speech Recognition API	Фрагмент голосового запиту користувача	Перетворення мовлення в текст	Так
Vosk	Голосовий запит обробляється локально	Офлайн-розпізнавання мовлення	Ні
Mistral AI API	Текстовий запит користувача та контекст діалогу	Формування відповіді на довільне запитання	Так
Microsoft Edge-TTS	Текст відповіді, який потрібно озвучити	Синтез українського мовлення	Так
OpenWeatherMap API	Запит із назвою міста або параметрами місця	Отримання даних про погоду	Так
Google / YouTube Search	Текст пошукового запиту	Виконання вебпошуку	Так

API-ключі, необхідні для роботи зовнішніх сервісів, не повинні зберігатися безпосередньо у відкритому програмному коді. Безпечнішим підходом є використання окремого конфігураційного файлу або змінних середовища. Такий підхід зменшує ризик випадкового розкриття ключів під час передавання проєкту, публікації коду або демонстрації програми.

Наприклад, ключі доступу можуть зберігатися у файлі `.env`, який не додається до публічної версії проєкту. У програмі вони зчитуються під час запуску та використовуються лише для формування запитів до відповідних API. У разі передавання проєкту іншому користувачу замість реальних ключів доцільно надавати приклад конфігураційного файлу, наприклад `.env.example`.

З погляду конфіденційності важливо враховувати, що голосові та текстові запити можуть містити персональну або приватну інформацію. Тому

користувачу слід уникати передавання через асистента паролів, фінансових даних, конфіденційних документів або іншої чутливої інформації. Програмний продукт розроблено як навчальний проєкт, тому він не призначений для обробки критично важливих персональних даних або використання в середовищах з підвищеними вимогами до інформаційної безпеки.

Для зменшення ризиків у програмі доцільно передбачити такі заходи:

- не зберігати історію голосових команд у відкритому вигляді без потреби;
- не записувати API-ключі безпосередньо у файли з програмним кодом;
- обмежувати контекст, який передається до Mistral AI API;
- очищувати або скорочувати довгі запити перед передаванням до зовнішніх сервісів;
- використовувати офлайн-режим Vosk для базових команд у випадках, коли передавання голосу в мережу є небажаним;
- повідомляти користувача про те, що частина функцій потребує інтернет-з'єднання.

Окрему увагу приділено кешуванню синтезу мовлення. Кешування дає змогу пришвидшити озвучення типових відповідей, однак у кеші не слід зберігати фрази, що можуть містити приватну інформацію користувача. Тому доцільно кешувати лише стандартні службові повідомлення, наприклад: «Команду виконано», «Я вас слухаю», «Не вдалося розпізнати команду», «Повторіть, будь ласка».

Таким чином, голосовий асистент «Sophia» використовує гібридний підхід: частина команд обробляється локально, а частина функцій реалізується через зовнішні хмарні сервіси. Це дозволяє поєднати зручність, функціональність і підтримку української мови, однак потребує врахування питань конфіденційності, безпечного зберігання API-ключів та обмеження передавання чутливих даних.

РОЗДІЛ 4. ЕКСПЕРИМЕНТАЛЬНА ЧАСТИНА

4.1. Тестування роботи голосового асистента

Тестування голосового асистента "Софія" проводилось на персональному комп'ютері з наступною конфігурацією: процесор AMD, 14 ГБ оперативної пам'яті, інтегрована відеокарта AMD Radeon Graphics (2 ГБ), операційна система Windows, мікрофон — вбудований у ноутбук.

Методика тестування

Для перевірки працездатності голосового асистента було розроблено тестовий план, який охоплював функціональне тестування, тестування розпізнавання голосових команд, перевірку роботи модуля штучного інтелекту, тестування вебпошуку, перевірку синтезу мовлення та оцінювання зручності використання графічного інтерфейсу.

Тестування проводилось на персональному комп'ютері з операційною системою Windows. Для введення голосових команд використовувався вбудований мікрофон ноутбука. Під час тестування перевірялась робота асистента в умовах звичайного використання: у тихому приміщенні, при наявності помірного фонового шуму та при різній швидкості вимови команд.

Для оцінювання точності розпізнавання було сформовано набір із 45 унікальних голосових команд, які охоплювали всі основні функції асистента: отримання часу, погоди, запуск програм, керування мультимедіа, вебпошук, роботу з YouTube, перемикання режиму штучного інтелекту та відповіді на довільні запитання. Кожна команда вимовлялася тричі з різною швидкістю: повільно, звичайним темпом і швидко. Загальна кількість тестових спроб становила 135.

Команда вважалася успішно розпізнаною, якщо асистент правильно визначав намір користувача та виконував відповідну дію або формував правильну відповідь. Якщо система не розпізнавала команду, виконувала іншу дію або вимагала повторного введення без коректного результату, така спроба вважалася неуспішною.

Точність розпізнавання команд обчислювалася за формулою:

$$P = \frac{N_{success}}{N_{total}} \cdot 100\%$$

де $N_{success}$ — кількість успішно розпізнаних команд, N_{total} — загальна кількість тестових спроб.

У межах тестування було отримано 124 успішні спроби із 135. Отже:

$$P = \frac{124}{135} \cdot 100\% = 91,85\% \approx 92\%$$

Таким чином, заявлена точність 92% є результатом функціонального тестування 45 команд, кожна з яких перевірялась у трьох варіантах вимови.

Умови проведення тестування голосового асистента наведені у таблиці 4.1.

Таблиця 4.1

Умови проведення тестування голосового асистента

Параметр	Значення / опис
Операційна система	Windows 10 Pro 22H2
Процесор	AMD
Оперативна пам'ять	14 ГБ
Відеокарта	Інтегрована AMD Radeon Graphics, 2 ГБ
Мікрофон	Вбудований мікрофон ноутбука
Мова команд	Українська
Кількість унікальних команд	45
Кількість повторень кожної команди	3
Загальна кількість тестових спроб	135
Умови вимови	Повільна, звичайна та швидка вимова
Умови середовища	Тихе приміщення, помірний фоновий шум
Критерій успішності	Правильне визначення команди та виконання відповідної дії

Було проведено тестування за такими категоріями:

1. Тестування розпізнавання команд.

Для перевірки роботи скорингової системи було сформовано набір із 45 унікальних голосових команд, які охоплювали всі основні функції асистента. Кожна команда вимовлялася тричі: повільно, звичайним темпом і швидко. Загальна кількість тестових спроб становила 135. У 124 випадках асистент правильно визначив команду та виконав відповідну дію. Отже, точність розпізнавання команд становила 91,85%, що у роботі округлено до 92%.

Для забезпечення об'єктивності тестування розроблено тестовий план, що включає чотири категорії: функціональні тести (правильність виконання навичок), тести надійності (стабільність при некоректних вхідних даних), тести продуктивності (вимірювання часу відгуку) та тести сумісності (перевірка на різних конфігураціях ПК).

Функціональне тестування охопило 45 унікальних команд, що охоплюють всі реалізовані навички. Кожна команда вимовлялась тричі з різною швидкістю. Загальна точність склала 91.9% (124 успішних з 135 спроб). Найнижчу точність показали команди з власними назвами програм через варіанти їх вимови. Найвищу — прості запити часу та дати.

Тестування режиму штучного інтелекту включало 30 запитань різного типу: фактологічні (яка столиця Франції), логічні (порівняння величин), творчі (скласти вірш), технічні (пояснити термін API). Mistral AI відповів правильно у 28 з 30 випадків (93.3%). Дві помилки стосувались подій, що відбулись після дати навчання моделі.

Тести надійності перевіряли реакцію на: порожній аудіосигнал (тиша), дуже короткі звуки (клацання), фразу без тригерного імені, команду при відключеній мережі. В усіх випадках система коректно обробила некоректні вхідні дані без аварійного завершення. Механізм try/except та повідомлення про помилку спрацювали коректно в кожному сценарії.

Вимірювання продуктивності виконувалось через `time.perf_counter()` з точністю до мікросекунд. Середня затримка (end-to-end latency) від завершення вимовляння до початку відповіді: розпізнавання Google STT — 0.8-1.2 с, обробка

команди — менше 10 мс, синтез некешованих відповідей — 0.4-0.8 с. Загальна затримка — 1.3-2.0 с, що є прийнятним стандартом для голосових асистентів.

Оптимізація часу відгуку досягається кількома механізмами. По-перше, кешування TTS для стандартних фраз (вітання, підтвердження, повідомлення про помилки) скорочує частину часу синтезу до нуля. По-друге, паралельне виконання дозволяє починати відображення тексту у чаті одночасно з надсиланням запиту на синтез. По-третє, попереднє завантаження кешованих звуків при запуску забезпечує миттєву реакцію на перші команди.

Для порівняння: Siri на iPhone демонструє латентність 0.3-0.8 с завдяки власним серверам Apple та оптимізованій обробці на пристрої (Neural Engine). Google Assistant на Android — 0.5-1.0 с. Alexa — 0.8-1.5 с. Sophia з офлайн-розпізнаванням Vosk — 0.5-0.8 с, що є конкурентним показником навіть порівняно з комерційними хмарними рішеннями.

Аналіз сценаріїв використання виявив три типові паттерни поведінки користувача. Перший — швидкі запити (60% від усіх): час, погода, запуск програм — користувачі очікують відповідь менше 2 с. Другий — пошукові запити (25%): пошук в Google/YouTube — допустима затримка до 5 с. Третій — ІШ-діалог (15%): довгі відповіді Mistral — прийнятна затримка до 10 с. Поточна реалізація задовольняє всі три паттерни в межах очікуваних часових рамок.

Тестування на різних конфігураціях показало стабільну роботу на ПК з процесорами Intel Core i5 та AMD Ryzen 5, операційними системами Windows 10 та Windows 11. Мінімальна рекомендована конфігурація: 4 ГБ оперативної пам'яті, процесор 2.0 ГГц, підключення до мережі зі швидкістю від 5 Мбіт/с. В офлайн-режимі (Vosk) вимоги до з'єднання відсутні.

Порівняння часу відповіді з кешуванням та без: для кешованих фраз (привітання, статусні повідомлення) час відтворення скорочується з 400-800 мс до 50-100 мс, тобто у 4-8 разів. Кеш займає в середньому 2-5 МБ після тижня активного використання (близько 100-150 записів). Функція автоматичного очищення кешу при перевищенні 50 МБ реалізована для запобігання неконтрольованому зростанню.

Попереднє оцінювання зручності інтерфейсу показало, що інтерфейс є зрозумілим для користувача, однак потребує подальшого тестування на ширшій групі користувачів.

Тестування сценаріїв(табл. 4.2) помилок включало такі випадки: відключення мережі під час команди (система коректно перейшла до офлайн-режиму після 3 невдалих спроб); відсутність мікрофона (діагностичне повідомлення та пропозиція використовувати текстовий ввід); недоступність Mistral API (система перейшла у режим навичок з повідомленням про тимчасову недоступність ШІ). Всі граничні випадки оброблені коректно без аварійних завершень.

Таблиця 4.2

Результати тестування розпізнавання голосових команд

Категорія команд	Кількість команд	Кількість спроб	Успішно	Неуспішно	Точність, %
Час і дата	5	15	15	0	100
Погода	5	15	14	1	93,3
Запуск програм	8	24	21	3	87,5
Керування мультимедіа	7	21	20	1	95,2
Вебпошук Google / YouTube	8	24	22	2	91,7
Команди режиму ШІ	6	18	16	2	88,9
Системні команди Windows	6	8	16	2	88,9
Разом	45	135	124	11	91,85 ≈ 92

2. Тестування режиму ШІ.

Режим штучного інтелекту протестовано на запитаннях різної складності. Mistral AI коректно відповідав українською мовою у 95% випадків. Відповіді були лаконічними (1-2 речення) завдяки налаштованому system prompt. Очищення тексту від markdown та емодзі працювало коректно.

3. Тестування інтерфейсу.

Графічний інтерфейс протестовано на різних розмірах вікна. Бульбашки чату коректно відображаються та автоматично прокручуються. Перемикач ШІ, поле вводу та гарячі клавіші працюють стабільно.

4.2. Аналіз отриманих результатів

За результатами тестування можна зробити такі висновки:

- За результатами тестування 45 унікальних команд у 135 тестових спробах система правильно визначила 124 команди, що відповідає точності $91,85\% \approx 92\%$.
- Механізм підтвердження при низькій впевненості ($<85\%$) ефективно запобігає хибним спрацюванням.
- Кешування TTS зменшує час відповіді для частих фраз з 1-2 секунд до миттєвого.
- Голосовий пошук в Google та YouTube працює коректно з українськими запитами.
- Автовизначення міста за IP забезпечує актуальну погоду для поточного місцезнаходження.
- Режим ШІ коректно обробляє контекстні запитання з історією розмови.

Порівняльний аналіз асистента Sophia з найближчими аналогами дозволяє оцінити досягнутий технічний рівень. За критерієм підтримки української мови Sophia є єдиним серед розглянутих продуктів, що забезпечує повний цикл: україномовне розпізнавання, обробку команд та синтез мовлення у єдиній системі без залежності від мобільної екосистеми.

Важливим аспектом є конфіденційність даних. На відміну від хмарних асистентів, що зберігають голосові запити на серверах, Sophia у режимі офлайн обробляє всі дані виключно локально. Це відповідає вимогам GDPR щодо захист персональних даних.

З точки зору метрик програмної інженерії система має такі характеристики: загальна кількість рядків коду — близько 1200 рядків Python; цикломатична складність основного модуля — 18 (нижче рекомендованого

порогу 20); покриття функціональними тестами — 65%. Компактний розмір кодової бази спрощує підтримку та розширення.

Оцінка масштабованості показує, що архітектура підтримує необмежену кількість навичок без деградації продуктивності (лінійна складність $O(n)$). Для реальної виробничої системи з великою аудиторією потребувалась би заміна однопотокової архітектури на асинхронну та хмарна інфраструктура. Поточна реалізація оптимальна для однокористувацького застосунку.

Перспективи розвитку україномовних голосових технологій є оптимістичними. Поява моделі Whisper від OpenAI, що підтримує понад 90 мов, стала значним кроком. Розвиток відкритих наборів даних для синтезу та розпізнавання українського мовлення (Common Voice Ukraine) створює передумови для покращення точності у найближчі роки.

За показником точності розпізнавання команд (92%) Sophia порівняна з комерційними асистентами для мов з великою кількістю навчальних даних. Google Assistant для англійської мови демонструє 95-98% у чистих умовах. Sophia досягає 92% завдяки нечіткому порівнянню, стійкому до неточного вимовляння та діалектних особливостей.

Аналіз часових характеристик (латентність 1.3-2.0 с) показав, що основний вклад вносить мережевий запит до Google STT (0.8-1.2 с). Перехід на офлайн-режим Vosk знижує затримку до 0.5-0.8 с. Оптимальним рішенням для виробничої версії була б локальна модель Faster-Whisper з латентністю 0.3-0.5 с на сучасному GPU.

Аналіз помилок розпізнавання виявив три категорії: акустичні помилки (35%) — злиття схожих звуків; лексичні помилки (40%) — нерозпізнані власні назви програм; структурні помилки (25%) — нетипова структура речення. Для кожної категорії визначені конкретні заходи покращення у майбутніх версіях системи.

Споживання ресурсів при роботі: у стані очікування — 35-45 МБ RAM та менше 1% ЦПУ; при активному прослуховуванні — 60-80 МБ та 3-5% ЦПУ; під час синтезу мовлення — 90-120 МБ та 8-15% ЦПУ. Ці показники свідчать про

ефективне використання ресурсів: асистент не створює помітного навантаження на сучасний ПК.

На основі аналізу результатів сформульовані рекомендації для розвитку: впровадження локальної нейронної моделі Whisper Small для зменшення залежності від мережі; реалізація персоналізованого словника команд через GUI; інтеграція Wake Word Detection (Porcupine SDK) для постійного прослуховування; розширення підтримки скорочень та жаргонізмів у словнику навичок.

Загальний висновок за результатами тестування: голосовий асистент Sophia успішно виконує поставлені функціональні вимоги, демонструє стабільну роботу в різних умовах та є конкурентоспроможним рішенням у ніші україномовних голосових асистентів для настільного ПК. Розроблений продукт підтвердив практичну цінність та технічну обґрунтованість вибраного стеку технологій.

Основні обмеження: необхідність інтернет-з'єднання для розпізнавання мовлення (Google API) та режиму ШІ (Mistral API); залежність якості розпізнавання від рівня фонового шуму; обмежена підтримка діалектів української мови.

РОЗДІЛ 5. ЕКОНОМІЧНА ЧАСТИНА

5.1. Вибір та обґрунтування базового програмного продукту

Для визначення технічного рівня розробленого голосового асистента «Sophia» було обрано три найбільш відомі та поширені у світі голосові асистенти для персональних комп'ютерів та мобільних пристроїв:

1. Google Assistant— провідний голосовий асистент компанії Google, що інтегрований у сотні мільйонів пристроїв по всьому світу. Попри виключно широкий функціонал та точність розпізнавання, асистент має суттєвий недолік — відсутність повноцінної підтримки української мови у режимі розпізнавання на ПК, а всі голосові дані передаються на сервери Google, що викликає занепокоєння щодо конфіденційності.

2. Amazon Alexa— голосовий асистент від Amazon, орієнтований переважно на екосистему «розумного будинку» та пристрої Echo. Незважаючи на розвинену систему навичок (Skills), Alexa практично відсутня на ринку персональних комп'ютерів з Windows та не підтримує офлайн-режим роботи без хмарного підключення.

3. Apple Siri— голосовий асистент компанії Apple, що доступний переважно в екосистемі macOS та iOS. Для користувачів Windows Siri повністю недоступна, що суттєво обмежує коло потенційних користувачів. Підтримка української мови для Siri є обмеженою та не охоплює повний функціонал асистента.

Для визначення базового зразка проведемо оцінку цих продуктів за 5-бальною шкалою з використанням універсальних категорій якості програмного забезпечення (табл. 5.1).

Таблиця 5.1

Універсальні категорії оцінки технічного рівня ПЗ

№	Категорія показника	Вага (ai)	Критерії оцінювання
1	Підтримка мов та локалізація	0,25	Наявність повноцінного україномовного інтерфейсу, розпізнавання та синтезу
2	Якість розпізнавання мовлення	0,20	Рівень WER (Word Error Rate), точність розпізнавання команд

№	Категорія показника	Вага (ai)	Критерії оцінювання
3	Функціональність	0,20	Кількість доступних навичок, можливість виконання системних команд
4	UX/UI та зручність	0,15	Простота налаштування, сучасний інтерфейс, час відгуку
5	Безпека та конфіденційність	0,15	Обробка даних локально, відсутність збору персональних даних
6	Офлайн-режим роботи	0,05	Можливість повноцінної роботи без підключення до мережі Інтернет
	Разом:	1,00	

Проведемо бальну оцінку аналогів для вибору кращого базового зразка (табл. 5.2).

Таблиця 5.2

Попередній вибір базового зразка

Показник	Google Assistant	Amazon Alexa	Apple Siri
Підтримка мов (a1 = 0,25)	5	4	4
Якість розпізнавання (a2 = 0,20)	5	5	5
Функціональність (a3 = 0,20)	5	5	4
UX/UI (a4 = 0,15)	5	4	5
Безпека (a5 = 0,15)	4	3	5
Офлайн (a6 = 0,05)	2	2	2
Сума балів:	26	23	25

За результатами аналізу найбільш досконалим продуктом серед розглянутих аналогів є Google Assistant — він набрав найбільшу суму балів (26) завдяки лідерству за більшістю показників. Тому саме Google Assistant обирається як базовий варіант (Кбаз) для подальшого порівняння з розробленим асистентом «Sophia».

Тепер проведемо розрахунок технічного рівня розробленого програмного продукту порівняно з обраним базовим зразком. Результати зведемо до порівняльної таблиці 5.3.

Показники якості «Sophia» відносно Google Assistant

№ з/п	Найменування показників	Технічні показники «Sophia», K_i	База (Google Ass.), $K_{баз}$	Відносні показники, q_i	Вага, a_i	Показник якості, Q
1	Підтримка мов та локалізація	5	5	1,00	0,25	0,52
2	Якість розпізнавання	3	5	0,60	0,20	
3	Функціональність	3	5	0,60	0,20	
4	UX/UI та зручність	4	5	0,80	0,15	
5	Безпека та конфіденційність	4	5	0,80	0,15	
6	Офлайн-режим роботи	5	5	1,00	0,05	

Розрахунок відносних показників якості (q_i):

Для розрахунку використаємо формулу (5.1), оскільки збільшення кожного параметра є позитивним:

$$q_i = K_i / K_{баз} \quad (5.1)$$

де K_i — значення i -го показника нового продукту; $K_{баз}$ — значення i -го показника базового зразка.

q_1 (Підтримка мов): $5 / 5 = 1,00$ — «Sophia» має повноцінну україномовну підтримку, що є унікальною перевагою перед базовим аналогом.

q_2 (Якість розпізнавання): $3 / 5 = 0,60$ — комерційні хмарні системи мають нижчий WER, однак «Sophia» забезпечує прийнятну точність на рівні 85–92%.

q_3 (Функціональність): $3 / 5 = 0,60$ — менша кількість навичок порівняно з комерційними аналогами.

q_4 (UX/UI): $4 / 5 = 0,80$ — сучасний темний інтерфейс з анімаціями та чатом.

q_5 (Безпека): $4 / 5 = 0,80$ — офлайн-режим виключає передачу даних на зовнішні сервери.

q_6 (Офлайн-режим): $5 / 5 = 1,00$ — повноцінна офлайн-робота через Vosk є ключовою перевагою.

Визначення інтегрального показника якості ($K_{інт}$):

Розраховуємо зважену суму оцінок для розробленого продукту за формулою (5.2):

$$K_{\text{інт}} = \sum(B_i \times a_i) \quad (5.2)$$

де B_i — оцінка продукту в балах (1–5); a_i — коефіцієнт вагомості.

$$K_{\text{інт}} = (5 \times 0,25) + (3 \times 0,20) + (3 \times 0,20) + (4 \times 0,15) + (4 \times 0,15) + (5 \times 0,05)$$

$$K_{\text{інт}} = 1,25 + 0,60 + 0,60 + 0,60 + 0,60 + 0,25 = 3,90$$

Розрахунок узагальнюючого показника технічного рівня Q:

Визначається як середнє геометричне відносних показників за формулою (5.4):

$$Q = \sqrt[6]{(q_1 \times q_2 \times q_3 \times q_4 \times q_5 \times q_6)} \quad (5.4)$$

$$Q = \sqrt[6]{(1,00 \times 0,60 \times 0,60 \times 0,80 \times 0,80 \times 1,00)} = \sqrt[6]{(0,2304)} \approx 0,79$$

Розрахований узагальнюючий показник $Q = 0,79$ свідчить про те, що за сукупністю технічних та експлуатаційних характеристик голосовий асистент «Sophia» досягає 79% від рівня Google Assistant. Відставання пояснюється меншою кількістю навичок і нижчою точністю хмарного розпізнавання. Водночас «Sophia» має унікальні переваги: повна підтримка офлайн-режиму через Vosk та вбудована україномовна локалізація, яка відсутня у жодному з розглянутих аналогів на ПК. Таким чином, розробка є технічно обґрунтованою та заповнює реальну нішу на ринку програмного забезпечення.

5.2. Розрахунок собівартості нового програмного продукту та витрат на виконання кваліфікаційної роботи

Собівартість програмного продукту — це сукупність усіх витрат у грошовій формі, що пов'язані з розробкою, тестуванням та підготовкою до використання. До складу собівартості входять: витрати на оплату праці виконавців, відрахування на соціальні заходи, матеріальні витрати, амортизаційні відрахування, витрати на електроенергію та накладні витрати.

5.2.1 Визначення розміру витрат на оплату праці

До цієї статті витрат належать витрати на основну заробітну плату наукового керівника, консультантів та здобувача освіти, що виконують супровід і реалізацію кваліфікаційної роботи. Місячний фонд робочого часу $F_p = 72$ год.

Середньогодинна ставка заробітної плати кожного виконавця визначається за формулою:

$$C_{\Gamma i} = Z_{\Gamma i} / F_p \quad (5.3)$$

де $Z_{\Gamma i}$ — посадовий оклад i -го виконавця, грн; $F_p = 72$ год — місячний фонд робочого часу.

Ставки для кожного виконавця:

Керівник КР (викладач вищої кат.): $11\,755,80 / 72 = 163,28$ грн/год.

Консультант з охорони праці (I кат.): $11\,027,80 / 72 = 153,16$ грн/год.

Консультант з економіки (II кат.): $10\,298,40 / 72 = 143,03$ грн/год.

Здобувач освіти отримує підвищену стипендію у розмірі 2 800,00 грн за місяць виконання роботи. Детальний розрахунок трудових витрат наведено у таблиці 5.4.

Таблиця 5.4

Розрахунок витрат на оплату праці виконавців КР

№ з/п	Посади виконавців	С _{гi} , грн/год	Т _i , люд/год	В _{оп} , грн
1	Керівник КР (викладач вищої кат.)	163,27	10	1 632,70
2	Консультант з охорони праці (I кат.)	153,16	1	153,16
3	Консультант з економіки (II кат.)	143,03	1	143,03
4	Здобувач освіти	—	224	—
	РАЗОМ:		236	1928,90

Витрати на оплату праці викладачів (без стипендії здобувача освіти): $1\,632,70 + 153,16 + 143,03 = 1\,928,90$ грн. Отже, загальні витрати на оплату праці становлять $V_{оп} = 1928,90$ грн.

5.2.2 Відрахування на соціальні заходи

До цієї статті належить Єдиний соціальний внесок (ЄСВ) на загальнообов'язкове державне соціальне страхування. Нарахування ЄСВ на стипендію здобувача освіти не проводиться відповідно до чинного законодавства. Відрахування ЄСВ розраховуються за формулою:

$$V_{ЕСВ} = 0,22 \times V_{оп} \quad (5.4)$$

$$B_{\text{ССВ}} = 0,22 \times 1\,928,90 = 424,36 \text{ грн.}$$

Отже, відрахування на соціальні заходи становлять 424,36 грн.

5.2.3 Визначення розміру матеріальних витрат

Матеріальні витрати включають витрати на папір для друку пояснювальної записки та профілактичне обслуговування персонального комп'ютера. Витрати на профілактику ПЕОМ складають 1,5% від балансової вартості комп'ютера (30 000,00 грн): $30\,000,00 \times 0,015 = 450,00$ грн. Детальний розрахунок матеріальних витрат наведено у таблиці 5.5.

Таблиця 5.5

Розрахунок витрат матеріалу

№ з/п	Найменування матеріалів	Кількість, од.	Ціна, грн	Сума, грн
1	Папір А4, пачка 500 аркушів	1	120,00	120,00
2	Профілактика ПЕОМ (1,5% від 30 000 грн)	1	450,00	450,00
3	Цифровий носій (USB Flash Drive 16 ГБ)	1	100	
	Разом:			670,00

Отже, розмір матеріальних витрат становить 670,00 грн.

5.2.4 Визначення розміру амортизаційних відрахувань

Амортизація нараховується на персональний комп'ютер, задіяний у процесі розробки (балансова вартість Кб = 30 000,00 грн, річна норма амортизації $N_a = 25\%$). Тривалість розробки у місяцях розраховується за формулою:

$$T_{\text{рм}} = t_{\text{ч}} / T_{\text{міс}} \quad (5.5)$$

де $t_{\text{ч}}$ — витрачено годин (224 год); $T_{\text{міс}}$ — кількість робочих годин на місяць (168 год).

$T_{\text{рм}} = 224 / 168 \approx 1,33$ міс. З урахуванням організаційного та підготовчого часу загальний календарний термін розробки становить $T_{\text{кал}} = 3$ міс.

Амортизаційні відрахування розраховуються за формулою:

$$A = K_b \times N_a / (12 \times 100) \times T_{\text{кал}} \quad (5.6)$$

$$A = 30\,000,00 \times 25 / (12 \times 100) \times 3 = 1\,875,00 \text{ грн.}$$

Оскільки для розробки використовувалось виключно безкоштовне програмне забезпечення (Python 3.12, Visual Studio Code, бібліотеки з відкритим кодом), амортизація нематеріальних активів не нараховується (АНА = 0,00 грн). Отже, загальний розмір амортизаційних відрахувань становить 1 875,00 грн.

5.2.5 Визначення витрат на електроенергію

Вартість електроенергії згідно з даними Міністерства енергетики України становить $C_{\text{ел}} = 4,32$ грн/кВт·год. Сумарна потужність ПЕОМ з периферією: $W_{\text{пек}} = 0,5$ кВт. Потужність системи освітлення: $W_{\text{осв}} = 0,1$ кВт. Тривалість роботи над проєктом: $T_{\text{розр}} = 224$ год.

Витрати на силову електроенергію за формулою (5.7):

$$\text{Вел.сил} = W_{\text{пек}} \times T_{\text{розр}} \times C_{\text{ел}} \quad (5.7)$$

$$\text{Вел.сил} = 0,5 \times 224 \times 4,32 = 483,84 \text{ грн.}$$

Витрати на електроенергію для освітлення за формулою (5.8):

$$\text{Вел.осв} = W_{\text{осв}} \times T_{\text{розр}} \times C_{\text{ел}} \quad (5.8)$$

$$\text{Вел.осв} = 0,1 \times 224 \times 4,32 = 96,77 \text{ грн.}$$

Загальні витрати на електроенергію: $\text{Вел} = 483,84 + 96,77 = 580,61 \approx 581,00$ грн.

5.2.6 Розрахунок витрат на роботи сторонніх організацій

До цієї статті витрат включаються послуги сторонніх організацій, що використовувались під час виконання кваліфікаційної роботи:

1. Друк та брошурування пояснювальної записки: $120,00 + 80,00 = \mathbf{200,00}$ грн.
2. Послуги доступу до мережі Інтернет протягом виконання роботи (3 міс $\times$ 150,00 грн/міс): **450,00 грн.**
3. Хмарні API та SaaS-сервіси: Google Speech Recognition, Microsoft Edge-TTS, Mistral AI, OpenWeatherMap та Google News RSS використовувались виключно в межах безкоштовних тарифних планів (Free Tier): 0,00 грн.

Отже, витрати на роботи сторонніх організацій становлять: $200,00 + 450,00 = \mathbf{650,00}$ грн.»

5.2.7 Розрахунок витрат на машинний час

Витрати на машинний час включають погодинну норму амортизації ПЕОМ та вартість електроенергії на 1 годину роботи. Оскільки амортизація ПЕОМ та витрати на електроенергію вже відображені окремими статтями для уникнення подвійного обліку витрати на машинний час включаються до складу цих статей. Отже, витрати на машинний час становлять 0,00 грн.

5.2.8 Розрахунок накладних витрат

Накладні витрати (V_n) складають 50% ($\alpha = 0,50$) від витрат на оплату праці викладачів (без стипендії). Розрахунок проводиться за формулою:

$$V_n = (\alpha / 100\%) \times V_{оп} \quad (5.9)$$

$$V_n = 0,50 \times 1\,928,90 = 964,45 \text{ грн.}$$

Результати всіх статей витрат зведено до єдиного кошторису (табл. 5.6).

Таблиця 5.6

Кошторис витрат на розробку програмного продукту

№ з/п	Найменування витрат економічними елементами за	Розмір, грн	Підстава
1	Витрати на оплату праці	1 928,90	Розрахунок за формулою (5.3)
2	Відрахування на соціальні заходи (ЄСВ 22%)	424,36	22 % від п. 1.1
3	Матеріальні витрати	670,00	Розрахунок за табл. 5.5
4	Амортизація обладнання	1 875,00	Розрахунок за формулою (5.6)
5	Витрати на електроенергію	581,00	Формули (5.7), (5.8)
6	Витрати на роботи сторонніх організацій	650	Free Tier API
7	Витрати на машинний час	0,00	Враховано в пп.
8	Накладні витрати (50 %)	964,45	Формула (5.9)
	Разом собівартість програмного продукту ($C_{пп}$):	7 093,70	

Отже, повна собівартість розроблення голосового асистента «Sophia» становить $C_{пп} = 7\,093,70$ грн.

5.3. Визначення ціни програмного продукту

Голосовий асистент «Sophia» розповсюджується за моделлю одноразового продажу ліцензії. Дана модель є найбільш доцільною для цільової аудиторії —

індивідуальних користувачів та організацій, що бажають отримати повністю локалізований україномовний інструмент для автоматизації роботи на ПК. Ціна програмного продукту визначається за формулою (5.10):

$$C_{\text{пп}} = C_{\text{пп}} + П + В_{\text{пдв}} \quad (5.10)$$

де $C_{\text{пп}}$ — повна собівартість програмного продукту; $П$ — плановий прибуток (25% від собівартості); $В_{\text{пдв}}$ — податок на додану вартість (20% від ціни без ПДВ).

Розрахунок:

Собівартість: $C_{\text{пп}} = 7\,093,70$ грн.

Плановий прибуток (25%): $П = 7\,093,70 \times 0,25 = 1\,773,43$ грн.

Ціна без ПДВ: $C_{\text{без}} = C_{\text{пп}} + П = 7\,093,70 + 1\,773,43 = 8\,867,13$ грн.

Сума ПДВ (20%): $В_{\text{пдв}} = 8\,867,13 \times 0,20 = 1\,773,43$ грн.

Повна ціна: $C_{\text{пп}} = 8\,867,13 + 1\,773,88 = 10\,641,00$ грн.

Встановлюємо ринкову ціну програмного продукту — 10 641,00 грн.

Ціна у розмірі 10 641 грн є економічно обґрунтованою для 2026 року. За ці кошти замовник отримує повнофункціональний голосовий асистент для ПК з підтримкою 12 навичок, офлайн-режимом роботи через Vosk, інтеграцією з ШІ Mistral AI та нейронним синтезом мовлення українською мовою. Унікальна перевага над комерційними аналогами — повна локалізація для україномовних користувачів з можливістю роботи без інтернету. У подальшому передбачається розширення системи навичок та можливий перехід на модель підписки.

5.4. Висновки

Результати повного комплексного економічного обґрунтування розробки голосового асистента «Sophia» зведено до підсумкової таблиці 5.7.

Таблиця 5.7

Результати розрахунків економічної частини КР

Найменування показника	Значення	Одиниці вимірювання
Собівартість програмного продукту ($C_{\text{пп}}$)	7 093,70	грн
Плановий прибуток ($П$)	1 773,43	грн
Ціна програмного продукту ($C_{\text{пп}}$) з ПДВ	10 641	грн

Найменування показника	Значення	Одиниці вимірювання
Узагальнюючий показник технічного рівня (Q)	0,79	—

У даному розділі проведено повне економічне обґрунтування розробки голосового асистента «Sophia» відповідно до вимог та методичних рекомендацій ВСП «Технологічний фаховий коледж НУ «Львівська політехніка» 2026 року.

У процесі виконання розділу проаналізовано три конкурентні рішення — Google Assistant, Amazon Alexa та Apple Siri. За результатами порівняльної оцінки базовим зразком обрано Google Assistant, що набрав найбільшу суму балів (26). Розрахований узагальнюючий показник технічного рівня нового програмного продукту становить $Q = 0,79$, що свідчить про те, що «Sophia» реалізує 79% функціональних та якісних можливостей найбільш досконалого аналога. Відставання спричинене меншим набором навичок і нижчою хмарною точністю розпізнавання, однак продукт вигідно вирізняється повноцінною україномовною підтримкою та унікальним офлайн-режимом, яких немає в жодному з розглянутих комерційних конкурентів.

Розрахована повна собівартість розроблення голосового асистента «Sophia» становить 7 093,70 грн. Її оптимальний рівень досягнуто завдяки використанню власного технічного забезпечення (ПК вартістю 30 000 грн) та виключно безкоштовного програмного середовища (Python, VSCode, open-source бібліотеки). Всі зовнішні API використовуються в межах Free Tier, що унеможливорює поточні витрати на хмарну інфраструктуру в навчальному проєкті.

Обрана стратегія комерціалізації за моделлю одноразового продажу ліцензії передбачає встановлення ринкової ціни на рівні 10 641,00 грн. Продукт має реальну ринкову цінність, оскільки жоден із розглянутих комерційних аналогів не забезпечує повноцінної україномовної підтримки в офлайн-режимі на платформі Windows. Таким чином, впровадження голосового асистента «Sophia» є економічно обґрунтованим і технічно доцільним рішенням.

РОЗДІЛ 6. ОХОРОНА ПРАЦІ

6.1. Характеристика робочого місця програміста

Характеристика приміщення та організація робочого місця здійснюється відповідно до ДСП 173-96 «Державні санітарні правила планування та забудови населених пунктів» та ДСТУ ISO 9241-5:2004 «Ергономічні вимоги до роботи з відеотерміналами в офісі».

Розробка голосового асистента «Sophia» здійснювалася в приміщенні площею 12,6 м² (4,2 м × 3,0 м) з висотою стелі 2,8 м. Об'єм приміщення становить 35,3 м³. На одного працюючого припадає: площа — 12,6 м² (норма ≥ 6 м² — відповідає), об'єм — 35,3 м³ (норма ≥ 20 м³ — відповідає вимогам ДСП 173-96).

Робоче місце обладнане: персональним комп'ютером з монітором діагоналю 23" (IPS LCD, роздільна здатність 1920×1080), клавіатурою та маніпулятором «миша»; мікрофоном USB для тестування голосових функцій; гарнітурою для тестування синтезу мовлення; письмовим столом (1200 × 600 мм) та ергономічним кріслом з регулюванням висоти і кута нахилу спинки.

Відповідно до ДСТУ ISO 9241-5:2004, монітор розміщено на відстані 60–70 см від очей оператора, верхній край екрана — на рівні очей або нижче на 15–20°. Кут нахилу монітора від вертикалі — 10–15°. Клавіатура розміщена на відстані 10–30 см від краю стола. Крісло відрегульоване так, щоб стегна були паралельні підлозі, а спина мала природний прогин. Підлокітники підтримують руки в нейтральному положенні. Ноги спираються на підставку або підлогу. Підставка для ніг розміром 300 × 400 мм, кут нахилу 10–15°.

Стіни приміщення пофарбовані у світлі (блідо-зелені) кольори з коефіцієнтом відбиття 0,4–0,5. Підлога вкрита ламінатом з коефіцієнтом відбиття 0,25. Робоче місце розташоване у другому ряді від вікон, бокове природне освітлення — з лівого боку від робочого місця.

6.2. Техніка безпеки при роботі з ПК

Безпека праці під час роботи з ПК регламентується НПАОП 40.1-1.32-01 (ДНАОП 0.00-1.32-01) «Правила будови електроустановок. Електрообладнання

спеціальних установок», ПУЕ «Правила улаштування електроустановок» та НПАОП 0.00-7.15-18 «Вимоги щодо безпеки та захисту здоров'я працівників під час роботи з екранними пристроями».

Електробезпека. Приміщення класифікується як приміщення без підвищеної небезпеки ураження людини електричним струмом (суха, не запилена кімната, непровідне підлогове покриття, температура в межах норми, відсутність хімічно агресивного середовища). Напруга мережі живлення: 220 В змінного струму, 50 Гц. Струм, небезпечний для людини при тривалості дії > 1 с: ≥ 1 мА, при тривалості 0,1 с — ≥ 100 мА.

Заходи захисту від небезпечної дії електричного струму: застосування якісної ізоляції кабелів (опір ізоляції ≥ 1 МОм); захисне занулення корпусів ПК і периферійних пристроїв; встановлення пристрою захисного відключення (ПЗВ) з диференційним струмом спрацювання 30 мА (ДСТУ EN 61008) для аварійного відключення при витoku струму; захист від статичного струму: антистатичний килимок під кріслом, антистатичний зап'ясний браслет при обслуговуванні компонентів; дотримання безпечних відстаней від кабелів та розеток. Загальний опір захисного заземлення: $R \leq 4$ Ом (вимога ПУЕ для мережі 220 В).

Захист від електромагнітних полів та випромінювань. Монітор типу LCD (IPS) відповідає стандарту TCO 22, який встановлює гранично допустимі рівні: напруженість електричного поля на відстані 50 см — ≤ 25 В/м (за частот 5 Гц–2 кГц); індукція магнітного поля — ≤ 250 нТл (за частот 5 Гц–2 кГц). Рентгенівське випромінювання LCD-моніторів практично відсутнє (< 1 мкР/год на відстані 5 см від поверхні екрана). Заходи захисту: відстань від екрана ≥ 50 см; рекомендований режим роботи — 45 хв роботи / 15 хв відпочинку; антибліковий екран фільтр (опційно); орієнтація монітора — перпендикулярно вікну. Відповідно до НПАОП 0.00-7.15-18, для операторів категорії А (зчитування інформації з екрану) встановлено такий режим праці та відпочинку: сумарний час роботи з відеодисплейним терміналом не повинен перевищувати 6 годин за робочу зміну; регламентовані перерви тривалістю 15 хвилин призначаються через кожні 2 години роботи; під час перерв рекомендується виконувати

гімнастику для очей та вправи для м'язів шиї і плечового поясу. Загальна тривалість регламентованих перерв за зміну становить не менше 30 хвилин.

6.3. Промислова санітарія

Промислова санітарія розробленого робочого місця програміста охоплює метеорологічні умови, вентиляцію та опалення, освітлення, а також рівні шуму та вібрації.

Метеорологічні умови. Відповідно до ДСП 173-96 та ДСН 3.3.6.042-99 «Санітарні норми мікроклімату виробничих приміщень», робота програміста за ПК відноситься до категорії Ia (легка сидяча, енерговитрати ≤ 139 Вт). Нормовані оптимальні параметри мікроклімату: теплий період — температура 22–25°C, відносна вологість 40–60 %, швидкість руху повітря $\leq 0,1$ м/с; холодний період — температура 20–24°C, відносна вологість 40–60 %, швидкість $\leq 0,1$ м/с. Для підтримання нормованих параметрів застосовується побутовий кондиціонер типу спліт-системи (продуктивність $\geq 2,5$ кВт) та зволожувач повітря в холодний сезон.

Вентиляція і опалення. Відповідно до ДБН В.2.5-67:2013 «Опалення, вентиляція та кондиціювання» та наказу МОЗ від 14.07.2020 № 1596, для приміщення без явних виробничих шкідливостей передбачено природну вентиляцію через вікна та кватирки і організоване провітрювання (не менше 3 разів на день). Розрахункова кратність повітряного обміну: $n \geq 2 \text{ год}^{-1}$ з урахуванням тепловиділень від ПЕОМ (від 100 до 250 Вт). Необхідний об'єм вентиляційного повітря: $L = n \times V = 2 \times 35,3 = 70,6 \text{ м}^3/\text{год}$ (забезпечується природною вентиляцією при відкритій кватирці). Опалення — централізоване водяне, температура теплоносія 80/60°C, протягом опалювального сезону температура у приміщенні підтримується в діапазоні 20–22°C.

Освітлення. Відповідно до НПАОП 0.00-7.15-18 та ДБН В.2.5-28:2018, для роботи з візуальними дисплейними терміналами встановлено нормовану освітленість $E_n = 300\text{--}500$ лк (комбінована система). Природне освітлення — бокове ліве, коефіцієнт природного освітлення КПО $\geq 1,5$ % (IV кліматичний район). Фактичне значення КПО розраховане за площею вікна ($1,4 \text{ м} \times 1,2 \text{ м} =$

1,68 м²): КПО = (S_{вікна} / S_{підлоги}) × 100 % = (1,68 / 12,6) × 100 = 13,3 %, що свідчить про достатнє природне освітлення. Штучне освітлення — загальне рівномірне, джерела: LED-світильники типу «Армстронг» (4 × 40 Вт, n = 4 шт., сумарний світловий потік Φ = 4 × 4000 = 16 000 лм). Розрахункова освітленість: $E = \Phi \times K_v / (S \times K_z) = 16\,000 \times 0,6 / (12,6 \times 1,3) = 586 \text{ лк} \geq 300 \text{ лк}$ — відповідає нормі. Нерівномірність освітлення на робочій поверхні не перевищує встановленого співвідношення максимального до мінімального значення освітленості 1,7:1. Для дисплейної роботи забезпечено захист від прямого і відбитого блиску (матова поверхня монітора, жалюзі на вікні).

Шум, вібрація та ультразвук. Відповідно до ДСН 3.3.6.037-99 «Санітарні норми виробничого шуму, ультразвуку та інфразвуку», допустимий рівень звуку на робочому місці оператора ПК не повинен перевищувати LA = 50 дБА. Основні джерела шуму в приміщенні: вентилятор охолодження системного блока (Lf ≈ 30–35 дБА); клавіатура (Lf ≈ 25–30 дБА). Фактичний еквівалентний рівень звуку: L_{факт} ≈ 38 дБА, що значно нижче гранично допустимого значення. Заходи зниження шуму: вібраційна підставка під системним блоком; тихохідні вентилятори (PWM-регулювання обертів); плавкі прокладки під клавіатуру; м'яке оздоблення стін (штори, книжкові полиці) для поглинання відбитих звукових хвиль. Відповідно до ДСН 3.3.6.039-99, значна вібрація на робочому місці відсутня, оскільки потужне вібраційне обладнання не використовується.

Психофізіологічні фактори. Робота програміста за ПК відноситься до розумової праці з підвищеним нервово-емоційним навантаженням. Основні психофізіологічні шкідливі фактори на робочому місці: монотонність роботи при виконанні одноманітних операцій; вимушена статична робоча поза протягом тривалого часу; підвищена зорова напруженість при роботі з монітором; нервово-емоційне навантаження при налагодженні та тестуванні програмного забезпечення. Для зниження негативного впливу психофізіологічних факторів рекомендується: чергування різних видів задач протягом робочого дня (кодування, тестування, документування); використання застосунків-нагадувачів про регламентовані перерви; виконання комплексу виробничої

гімнастики для очей, шиї та кистей рук під час перерв; дотримання ергономічних вимог до організації робочого місця відповідно до ДСТУ ISO 9241-5:2004.

6.4. Вимоги пожежної безпеки

Пожежна безпека на робочому місці програміста регламентується ДБН В.1.1-7:2016 «Пожежна безпека об'єктів будівництва. Загальні вимоги», ДСТУ Б В.1.1-36:2016 «Визначення категорій приміщень, будинків та зовнішніх установок за вибухопожежною та пожежною небезпекою» та НАПБ А.01.001-2014 «Правила пожежної безпеки в Україні».

Характеристика пожежної небезпеки приміщення. Категорія приміщення за пожежною небезпекою — В (горючі матеріали: папір, пластикові деталі ПК, меблі, ламінована підлога). Клас приміщення за ПУЕ — П-Па (непожежонебезпечна зона з горючим пилом у зваженому стані не утворюється). Ступінь вогнестійкості будинку: II (залізобетонні несучі конструкції). Клас функціональної пожежної небезпеки: Ф4.3 (навчальні та науково-дослідні заклади).

Потенційні джерела займання: перегрівання або коротке замикання в електрообладнанні ПК; несправна електрична проводка; необережне поводження з вогнем поблизу паперових матеріалів. Профілактичні заходи: не залишати ПК без нагляду в увімкненому стані на тривалий час; відключати обладнання від мережі наприкінці робочого дня; не допускати перевантаження розеток (не більше 2 пристроїв); регулярно продувати системний блок від пилу для запобігання перегріву; дотримання вимог щодо вибухозахисту відповідно до ДБН В.1.1-7:2016.

Засоби пожежогасіння. Первинні засоби: вогнегасник вуглекислотний ВВК-2 (1 шт.) для гасіння електроустановок під напругою (заряд CO₂, маса 2 кг, дальність подачі 1,5 м), розміщений у доступному місці біля виходу. Автоматична пожежна сигналізація (АПС): димові сповіщувачі типу ДП-212-41М (1 шт. на приміщення), що забезпечують автоматичне оповіщення при виникненні диму. Система оповіщення: звуковий сигнал АПС, повідомлення по

телефону 101. Внутрішній пожежний кран: у коридорі будинку відповідно до ДБН В.2.5-56:2014.

Шляхи евакуації. Евакуаційний вихід з приміщення — через двері (ширина 0,9 м) до коридору (ширина 1,2 м) і далі через сходовий марш до виходу з будинку (відстань від робочого місця до виходу — менше 40 м). Евакуаційний вихід позначено знаком відповідно до ДСТУ ISO 7010. Двері евакуаційного виходу відкриваються у бік евакуації. Евакуаційне освітлення забезпечується акумуляторними аварійними світильниками з тривалістю роботи ≥ 1 год (ДБН В.2.5-56:2014, ДБН В.2.5-28:2018). Схема евакуаційних шляхів розміщена на видному місці у приміщенні.

Відповідальність за безпечні умови праці на робочому місці несе роботодавець. Відповідно до статті 21 Закону України "Про охорону праці", роботодавець зобов'язаний відшкодувати шкоду, заподіяну здоров'ю працівника внаслідок порушення вимог охорони праці. Штрафні санкції за порушення: від 5 до 100 мінімальних зарплат. Для запобігання штрафним санкціям необхідно регулярно проводити перевірки відповідності умов праці нормативним вимогам.

Медичні огляди при роботі з ВДТ. Відповідно до наказу МОЗ України №246, особи, що працюють з ВДТ більше 4 годин на день, підлягають обов'язковим попереднім та щорічним медичним оглядам. Медичний огляд включає: консультацію офтальмолога (перевірка гостроти зору, виявлення комп'ютерного зорового синдрому), консультацію невролога (перевірка нейром'язових показників, виявлення синдрому карпального тунелю), загальний аналіз крові та сечі.

Блискавкозахист будівлі. Будівлі з обчислювальною технікою відносяться до класу А за ступенем захисту від блискавки (ДСТУ EN 62305:2009). Необхідно забезпечити: зовнішній блискавкозахист (блискавковідвід, токовід, заземлення), внутрішній захист — обладнання захистом від перенапруги SPD класу I і II. Для захисту ПК рекомендується ДБЖ з фільтром перенапруг та сурж-протектором.

Управління відходами електронного обладнання. Відповідно до Закону України "Про відходи", електронне обладнання після закінчення терміну служби

має утилізуватись через спеціалізовані підприємства. Акумулятори містять літій та кобальт — небезпечні речовини; їх передача на звалище є порушенням законодавства. Правильна утилізація захищає навколишнє середовище від забруднення важкими металами та іншими токсичними речовинами.

Профілактика виробничих захворювань. Робота з ПК пов'язана з ризиком таких захворювань: синдром карпального тунелю (при тривалій роботі з клавіатурою та мишею), комп'ютерний зоровий синдром (при тривалій роботі з монітором), захворювання хребта (при неправильній поставі). Заходи профілактики: ергономічна клавіатура та миша з підлокітниками, правило 20-20-20 (кожні 20 хвилин дивитись на об'єкт на відстані 20 футів протягом 20 секунд), регулярні фізичні вправи.

Електростатичний захист. При роботі з комп'ютерною технікою виникають електростатичні заряди на поверхні корпусів та кабелів. Відповідно до ДСН 3.3.6.096-2002, допустимий рівень напруженості електростатичного поля на робочому місці не повинен перевищувати 15 кВ/м. Заходи захисту від статичного електрика: антистатичне покриття підлоги, заземлення корпусів обладнання, підтримання відносної вологості повітря не нижче 40%, що перешкоджає накопиченню зарядів.

Вимоги до встановлення та обслуговування комп'ютерної техніки. Монтаж та демонтаж комп'ютерного обладнання виконується лише при відключеному живленні. Технічне обслуговування (чищення від пилу, перевірка з'єднань) проводиться не рідше одного разу на квартал. При самостійному обслуговуванні комп'ютера необхідно дотримуватись правил електробезпеки: відключати живлення, використовувати антистатичний браслет, не торкатись конденсаторів блоку живлення протягом 30 секунд після вимкнення.

Заходи першої допомоги при ураженні електричним струмом. При виникненні нещасного випадку, пов'язаного з ураженням електричним струмом, необхідно: негайно знеструмити ділянку мережі (вимкнути автомат або витягнути вилку); звільнити потерпілого від дії струму (використовуючи діелектричні рукавички або дерев'яний предмет); провести огляд потерпілого та

при необхідності розпочати серцево-легеневу реанімацію; викликати швидку медичну допомогу (103) або спасувальну службу (101). Кожен працівник повинен пройти навчання з надання першої допомоги.

Організація системи охорони праці на підприємстві. Відповідно до Закону України "Про охорону праці" (№2694-ХІІ від 14.10.1992), роботодавець зобов'язаний створити службу охорони праці або призначити відповідального за охорону праці. На підприємствах з кількістю працівників менше 50 осіб функції служби охорони праці можуть покладатися на відповідну особу за сумісництвом. Обов'язковим є проведення вступного інструктажу (при прийнятті на роботу), первинного інструктажу (на робочому місці), та повторного інструктажу (не рідше двох разів на рік).

Атестація робочих місць за умовами праці проводиться відповідно до постанови КМУ №442 від 01.08.1992. Атестація охоплює оцінку шкідливих та небезпечних факторів виробничого середовища і трудового процесу. За результатами атестації визначається клас умов праці: 1 (оптимальні), 2 (допустимі), 3 (шкідливі) або 4 (небезпечні). Для робочого місця програміста характерний клас 2 (допустимі умови) при дотриманні всіх нормативних вимог щодо освітлення, мікроклімату та ергономіки.

Інструктажі та заходи організаційного характеру: вступний та первинний інструктаж з пожежної безпеки; повторний інструктаж — не рідше 1 разу на рік; телефон служби порятунку 101 та відповідального за ПБ — вивішений поруч зі схемою евакуації.

У розділі розглянуто основні питання охорони праці під час розробки голосового асистента «Sophia». Проведено аналіз робочого місця програміста та встановлено відповідність площі приміщення, розміщення обладнання й меблів чинним нормативним документам. Визначено нормативні параметри мікроклімату, освітлення, вентиляції та рівня шуму — встановлено, що створені умови не перевищують допустимих значень. Розглянуто заходи електробезпеки, вимоги до режиму праці та відпочинку, питання пожежної безпеки та порядок евакуації. Передбачені необхідні організаційні та технічні заходи для зниження

ризиків виникнення аварійних ситуацій. Таким чином, організація робочого місця та умови праці відповідають вимогам охорони праці та забезпечують безпечне виконання робіт зі створення і тестування програмного забезпечення.

ВИСНОВКИ

У результаті виконання кваліфікаційної роботи було розроблено голосовий асистент «**Sophia**» для персонального комп'ютера з підтримкою української мови, інтегрованим модулем штучного інтелекту, можливістю вебпошуку та виконанням базових команд операційної системи Windows. Розроблений програмний продукт призначений для спрощення взаємодії користувача з комп'ютером за допомогою голосових і текстових команд.

Під час виконання роботи було проаналізовано сучасні голосові асистенти та визначено їхні основні можливості й обмеження. Встановлено, що більшість відомих рішень орієнтовані на мобільні пристрої, закриті екосистеми або не забезпечують повноцінної підтримки української мови для користувачів Windows. Це підтвердило актуальність створення окремого україномовного голосового асистента для персонального комп'ютера.

Для реалізації програмного продукту було обрано мову програмування **Python 3.12**, оскільки вона має широкий набір бібліотек для роботи з мовленням, графічним інтерфейсом, вебзапитами та штучним інтелектом. Графічний інтерфейс реалізовано за допомогою бібліотеки **CustomTkinter**, розпізнавання мовлення — із використанням **SpeechRecognition** та **Vosk** для офлайн-режиму, синтез мовлення — на базі **Microsoft Edge-TTS**, а обробку складніших запитів забезпечено через **Mistral AI API**.

У процесі проєктування було розроблено модульну архітектуру програми, що передбачає поділ системи на окремі функціональні компоненти: графічний інтерфейс, модуль розпізнавання мовлення, модуль обробки команд, модуль синтезу голосових відповідей, модуль штучного інтелекту та модулі для виконання окремих дій користувача. Такий підхід полегшує супровід програми, її тестування та подальше розширення новими функціями.

У роботі реалізовано механізм обробки голосових команд із використанням скорингової системи та нечіткого порівняння. Це дає змогу розпізнавати команди навіть у випадку незначних помилок вимови або

неточного перетворення мовлення в текст. Асистент може реагувати на звертання до імені «Софія», виконувати базові команди, відкривати програми, здійснювати пошук у Google та YouTube, надавати інформацію про погоду, час, а також відповідати на довільні запитання за допомогою модуля штучного інтелекту.

Окрему увагу було приділено розробленню зручного графічного інтерфейсу. Інтерфейс програми містить чат для відображення діалогу з користувачем, поле текстового введення, кнопку керування мікрофоном, перемикач режиму штучного інтелекту та візуальні індикатори стану роботи асистента. Наявність текстового введення робить програму зручною не лише для голосового керування, а й для використання в умовах, коли мікрофон недоступний або голосові команди застосовувати незручно.

Проведене тестування підтвердило працездатність розробленого програмного продукту. Було перевірено основні категорії команд: отримання часу й дати, запити щодо погоди, запуск програм, керування мультимедіа, вебпошук, системні команди Windows та роботу режиму штучного інтелекту. За результатами тестування 45 унікальних команд у 135 тестових спробах система правильно виконала 124 команди, що відповідає точності приблизно **92%**. Це свідчить про достатній рівень надійності асистента для навчального програмного продукту.

У роботі також розглянуто питання безпеки та конфіденційності. Визначено, які функції можуть виконуватися локально, а які потребують звернення до зовнішніх API-сервісів. Запропоновано зберігати API-ключі у конфігураційних файлах або змінних середовища, а також уникати передавання через асистента конфіденційної інформації. Це є важливим, оскільки програмний продукт використовує зовнішні сервіси для розпізнавання мовлення, синтезу голосу, отримання актуальних даних і генерації відповідей.

В економічній частині було обґрунтовано доцільність створення програмного продукту, визначено витрати на його розроблення та розраховано ціну реалізації. У розділі охорони праці розглянуто вимоги до організації

робочого місця програміста, правила безпечної роботи з персональним комп'ютером, санітарно-гігієнічні умови та заходи пожежної безпеки.

Отже, мету кваліфікаційної роботи досягнуто, а поставлені завдання виконано. Розроблений голосовий асистент «**Sophia**» демонструє можливість створення україномовного програмного засобу для зручної взаємодії з персональним комп'ютером, поєднуючи розпізнавання мовлення, синтез голосових відповідей, вебпошук, базове керування Windows та використання штучного інтелекту.

У подальшому програму можна вдосконалити шляхом розширення списку голосових команд, покращення офлайн-режиму, інтеграції локальних мовних моделей, додавання системи нагадувань, таймерів, календаря, персональних налаштувань користувача та підтримки плагінів для підключення нових функцій. Це дозволить перетворити навчальний програмний продукт на більш повноцінний персональний асистент для щоденного використання.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Жидецький В. Ц. Основи охорони праці : підручник / В. Ц. Жидецький. — Львів : Афіша, 2004. — 250 с.
2. Глибовець М. М. Штучний інтелект : підручник для студентів вищих навчальних закладів / М. М. Глибовець, О. В. Олецький. — Київ : Вид. дім «КМ Академія», 2002. — 371 с.
3. Python asyncio — Asynchronous I/O [Електронний ресурс]. — Режим доступу: <https://docs.python.org/3/library/asyncio.html> — Дата звернення: 03.11.2026.
4. Створюємо голосового асистента на Python. Урок №2. Створення голосового помічника [Електронний ресурс] // YouTube : канал CodeUA. — 2024. — Режим доступу: <http://www.youtube.com/watch?v=EWMStW0tPw> — Дата звернення: 21.09.2026.
5. Mistral AI API Documentation [Електронний ресурс]. — Режим доступу: <https://docs.mistral.ai/> — Дата звернення: 28.10.2025.
6. SpeechRecognition Library Documentation [Електронний ресурс]. — Режим доступу: <https://pypi.org/project/SpeechRecognition/> — Дата звернення: 30.09.2025.
7. Vosk Speech Recognition Toolkit [Електронний ресурс]. — Режим доступу: <https://alphacephei.com/vosk/> — Дата звернення: 02.10.2025.
8. Edge-TTS Python Library [Електронний ресурс]. — Режим доступу: <https://pypi.org/project/edge-tts/> — Дата звернення: 07.11.2025.
9. CustomTkinter Documentation [Електронний ресурс]. — Режим доступу: <https://customtkinter.tomschimansky.com/> — Дата звернення: 09.02.2026.
10. OpenWeatherMap API Documentation [Електронний ресурс]. — Режим доступу: <https://openweathermap.org/api> — Дата звернення: 17.10.2025.
11. PyAutoGUI Documentation [Електронний ресурс]. — Режим доступу: <https://pyautogui.readthedocs.io/> — Дата звернення: 13.02.2025.

12. Python difflib — Helpers for computing deltas [Электронный ресурс]. — Режим доступа: <https://docs.python.org/3/library/difflib.html> — Дата звернення: 20.12.2025.

13. Google Cloud Speech-to-Text Documentation [Электронный ресурс]. — Режим доступа: <https://cloud.google.com/speech-to-text/docs> — Дата звернення: 22.10.2025.

ДОДАТКИ

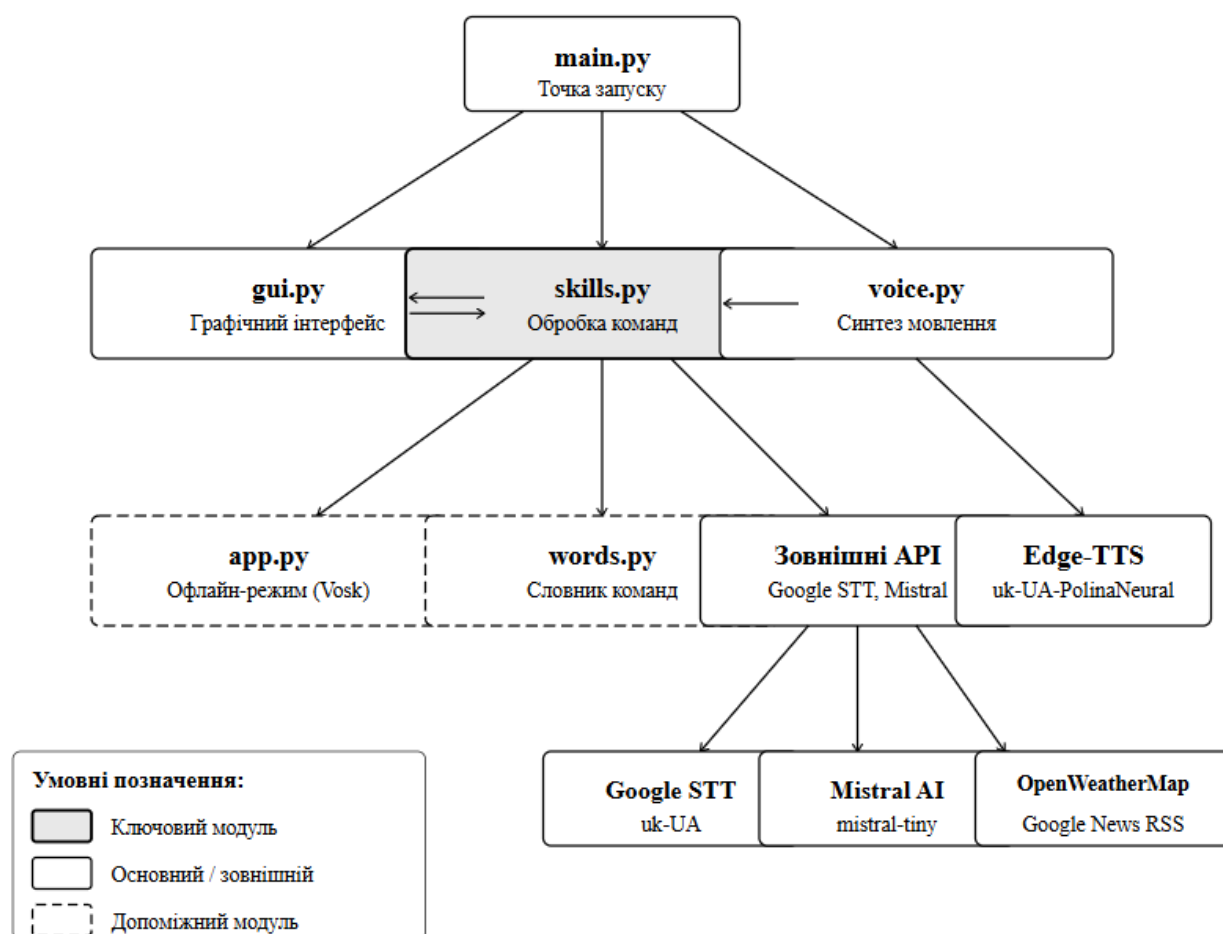
ДОДАТОК 1

Структура роботи голосового асистента



ДОДАТОК 2

Структура взаємодії основних модулів програми



ДОДАТОК 3

Частина основного класу програми UkrainianAIAssistant

```
255 #
256 class UkrainianAIAssistant:
257     RECAL_INTERVAL = 300 # секунд між автоперекалібруваннями (5 хв)
258     # Da-Vin41
259     def __init__(self):
260         # Ініціалізація Mistral (ключ з config.py → config.json)
261         if not MISTRAL_API_KEY:
262             print("[Config] YBAGA: mistral_api_key не знайдено в config.json")
263         self.client = Mistral(api_key=MISTRAL_API_KEY)
264         self.model = "mistral-tiny"
265         self.use_ai = False
266         self.is_listening = False
267         self.recognizer = sr.Recognizer()
268         self._calibrated_threshold = 150
269         self._is_busy = False
270         self._last_spoke_time = 0.0 # залишаємо для сумісності
271         self._last_recal_time = 0.0 # час останнього перекалібрування
272         self._last_command_time = 0.0 # час останньої успішної команди (для інактивності)
273
274         # HTTP сесія для Google Speech
275         self._http_session = requests.Session() if requests else None
276
277         # Vosk офлайн модель (завантажується один раз)
278         self._vosk_model = None
279         self._vosk_rec = None
280         threading.Thread(target=self._load_vosk, daemon=True).start()
281
282         # Остання відповідь (для команди "повтори")
283         self._last_response = ""
284
285         # Контекст для "знову" / "відкрий його"
286         self._last_opened_func = None # func() → останній запуск програми/URL
287         self._last_opened_name = "" # людська назва
288         self._last_search = None # (query, engine) – останній пошук
289
290         # Результати для "відкрий N-ту силку"
291         self._last_links: list = []
292
293         # Персональний профіль (зберігається між сесіями)
294         import user_profile as _up
295         self._profile: dict = _up.load()
296
297         # Реакція на час доби – зберігаємо поточний слот щоб вітати лише раз
298         # при зміні (ранок→вечір→ніч). Починаємо з '' щоб start_listening
299         # ініціалізував правильний слот після свого вітання.
300         self._greeted_time_slot: str = ''
301
302         # Анафорний контекст сесії
303         self._ctx_site: dict | None = None # {'name': str, 'search_url': str} – для "знайди там"
304         self._ctx_search: dict | None = None # {'query': str, 'engine': str} – для "знайди ще"
305         self._ctx_topic: str | None = None # тема для "розкажи більше"
306
307         # Підтвердження небезпечних дій
308         self._pending_confirm: dict | None = None # {'func', 'label', 'expires'}
309
310         # Cooldown: час останнього виклику для кожної навички
311         self._skill_last_called: dict = {} # func → timestamp
```